

Algorithmic Collusion in Financial Markets

Q-Learning Speculators in a Kyle Model with
Information-Insensitive Investors

Clemente Esquinas Coves

May 2026

1 Introduction

The rapid adoption of autonomous, model-free algorithms in financial markets has raised a concern: can self-interested algorithms learn to collude without explicit communication or pre-programmed intent? Recent work by Dou et al. (2025) suggests that the answer is yes, at least in controlled settings that preserve the essential informational structure of securities trading. This project revisits their framework, first characterizing the theoretical benchmarks analytically and then assessing whether Q-learning speculators trading against one another converge to collusive outcomes. The model extends the canonical Kyle (1985) setup with multiple symmetric informed speculators trading in parallel and a continuum of information-insensitive investors who trade mechanically against the price; the friction introduced by the latter is what opens the door to sustainable collusion when present.

The report proceeds in three parts. Section 3 derives the linear Nash and cartel equilibria, yielding closed-form expressions for trading intensity χ , price impact λ , and per-speculator profit at $\xi = 0$, and shows that the cartel earns higher per-speculator profit than Nash for $I \geq 2$. Section 4 develops the market-quality analysis: liquidity $\mathcal{L} = 1/\lambda$, price informativeness $\mathcal{I}$, and noise-trader losses W_N , with comparative statics in I at $\xi = 0$ and $\xi = 500$, and a discussion of the effective number of insiders I^* . Section 5 implements the Q-learning protocol of Dou et al. (2025), measures the collusion index Δ_χ in both regimes, and uses a noise-shock impulse response analysis to distinguish collusion sustained by price-trigger strategies from collusion sustained by over-pruning bias.

2 Model

Consider a repeated market for a risky asset with fundamental value $v_t \sim \mathcal{N}(\bar{v}, \sigma_v^2)$, drawn independently each period. There are four types of participants:

1. **Informed speculators.** There are $I \geq 1$ symmetric informed speculators. Each speculator i observes v_t at the beginning of period t and submits a market order $x_{i,t}$. Speculator i 's profit in period t is

$$\pi_{i,t} = (v_t - p_t)x_{i,t}.$$

2. **Noise trader.** A noise trader submits orders $u_t \sim \mathcal{N}(0, \sigma_u^2)$, independent of v_t and across periods.

3. **Information-insensitive investors.** Information-insensitive investors have aggregate demand

$$z_t = -\xi(p_t - \bar{v}), \quad \xi \geq 0. \tag{1}$$

When $\xi = 0$ the model reduces to the standard Kyle framework. When $\xi > 0$, these investors buy when the price is below $\bar{v}$ and sell when it is above, without inferring information from prices.

4. **Market maker.** The market maker observes the total order flow

$$y_t = \sum_{i=1}^I x_{i,t} + u_t$$

and the information-insensitive investor demand z_t . The market maker sets the price to minimize a weighted combination of pricing errors and inventory costs:

$$p_t = \arg \min_p \theta(p - \mathbb{E}[v_t | y_t])^2 + (y_t + z_t)^2, \quad (2)$$

where $\theta > 0$ governs the relative weight on price discovery.

The resulting linear pricing rule takes the form

$$p_t = \bar{v} + \lambda y_t. \quad (3)$$

3 Equilibrium Analysis

3.1 Pricing Rule

To derive the pricing rule, we solve the market maker's optimization problem (2) by minimizing the following expression with respect to p :

$$X(p) = \theta(p - \mathbb{E}[v_t | y_t])^2 + (y_t - \xi(p - \bar{v}))^2.$$

To find the optimal price, we differentiate X with respect to p and set it to zero:

$$\begin{aligned} \frac{dX}{dp} &= 2\theta(p^* - \mathbb{E}[v_t | y_t]) - 2\xi(y_t - \xi(p^* - \bar{v})) = 0 \iff \\ \iff \theta p^* - \theta \mathbb{E}[v_t | y_t] - \xi y_t + \xi^2 p^* - \xi^2 \bar{v} &= 0 \iff p^* = \frac{\xi^2 \bar{v}}{\theta + \xi^2} + \frac{\xi y_t}{\theta + \xi^2} + \frac{\theta \mathbb{E}[v_t | y_t]}{\theta + \xi^2}. \end{aligned}$$

Now, by assumption, $v_t = \bar{v} + \gamma y_t + \varepsilon_t$, where $\gamma = \mathbb{E}[v_t - \bar{v} | y_t]$ is the regression coefficient of v_t on y_t , and ε_t is uncorrelated with y_t . Therefore:

$$\mathbb{E}[v_t | y_t] = \mathbb{E}[\bar{v} | y_t] + \gamma \mathbb{E}[y_t | y_t] + \mathbb{E}[\varepsilon_t | y_t] = \bar{v} + \gamma y_t,$$

since $\bar{v}$ is a constant, y_t is y_t -known and ε_t is the residual of the regression which must have expectation zero. Also, $\mathbb{E}[\varepsilon_t | y_t] = \mathbb{E}[\varepsilon_t]$ because normality makes uncorrelatedness equivalent to independence: y_t is a linear combination of v_t and u_t (both Gaussian and independent), and ε_t is also a linear combination of v_t and u_t , so (ε_t, y_t) is jointly normal.

Hence, we get:

$$p^* = \frac{\xi^2 \bar{v}}{\theta + \xi^2} + \frac{\xi y_t}{\theta + \xi^2} + \frac{\theta(\bar{v} + \gamma y_t)}{\theta + \xi^2} = \bar{v} + \frac{\gamma \theta + \xi}{\theta + \xi^2} y_t = \bar{v} + \lambda y_t.$$

Additionally:

$$\frac{d^2 X}{dp^2} = 2\theta + 2\xi^2 > 0,$$

which confirms p^* achieves a minimum. This yields the pricing rule (3).

3.2 Nash Equilibrium

In the context of this model, a Nash equilibrium is a symmetric strategy such that no speculator can increase their expected profit by unilaterally deviating, taking the strategies of all other speculators as given. Here we derive speculator i 's best response and impose symmetry to characterize the Nash equilibrium.

Suppose each speculator i uses a linear strategy $x_{i,t} = \chi_i(v_t - \bar{v})$. Speculator i 's profit in period t is therefore $\pi_{i,t} = (v_t - p_t)\chi_i(v_t - \bar{v})$. Substituting the pricing rule (3):

$$\pi_{i,t} = (v_t - \bar{v} - \lambda y_t)\chi_i(v_t - \bar{v}).$$

Now, fixing the strategies of all speculators $j \neq i$ at intensity χ , we obtain $y_t = (I - 1)\chi(v_t - \bar{v}) + \chi_i(v_t - \bar{v}) + u_t$. Substituting into the expression for $\pi_{i,t}$:

$$\pi_{i,t} = \chi_i(v_t - \bar{v})(v_t - \bar{v} - \lambda(\chi(I - 1)(v_t - \bar{v}) + \chi_i(v_t - \bar{v}) + u_t)).$$

Expanding the expression for $\pi_{i,t}$:

$$\begin{aligned} \pi_{i,t} &= \chi_i(v_t - \bar{v})^2 - \lambda\chi_i(v_t - \bar{v})[\chi(I - 1)(v_t - \bar{v}) + u_t] - \lambda\chi_i^2(v_t - \bar{v})^2 = \\ &= (v_t - \bar{v})^2 [\chi_i - \lambda\chi_i\chi(I - 1) - \lambda\chi_i^2] - \lambda\chi_i(v_t - \bar{v})u_t. \end{aligned}$$

The best response is given by the expected profit $P(\lambda, \chi, \chi_i)$ conditioned on the fundamental value of the risky asset at time t , so:

$$P(\lambda, \chi, \chi_i) = \mathbb{E}[\pi_{i,t}(\lambda, \chi, \chi_i) \mid v_t] = (v_t - \bar{v})^2 [\chi_i - \lambda\chi_i\chi(I - 1) - \lambda\chi_i^2].$$

This last equality is due to $(v_t - \bar{v})$ being v_t -known and $\mathbb{E}[(v_t - \bar{v})u_t \mid v_t] = (v_t - \bar{v})\mathbb{E}[u_t] = 0$ (u_t is independent of v_t). Taking the first order condition of $P(\lambda, \chi, \chi_i)$ with respect to χ_i (where we assume $\lambda > 0$, as will be verified below):

$$\frac{\partial P}{\partial \chi_i} = (v_t - \bar{v})^2 [1 - \lambda\chi(I - 1) - 2\lambda\chi_i] = 0 \iff 1 - \lambda\chi(I - 1) - 2\lambda\chi_i^* = 0$$

$$\iff 2\lambda\chi_i^* = 1 - \lambda\chi(I - 1) \iff \chi_i^* = \frac{1}{2\lambda} - \frac{\chi(I - 1)}{2}.$$

Lastly, imposing the symmetry of the Nash equilibrium condition $\chi_i = \chi$:

$$\chi^N = \frac{1}{2\lambda^N} - \frac{\chi^N(I - 1)}{2} \iff \chi^N \left(1 + \frac{I - 1}{2}\right) = \frac{1}{2\lambda^N} \iff$$

$$\chi^N = \frac{1}{(I+1)\lambda^N}, \quad (4)$$

which is the first condition of the Nash equilibrium.

From the definition of λ :

$$\lambda^N = \frac{\theta\gamma^N + \xi}{\theta + \xi^2}. \quad (5)$$

Since γ is the regression coefficient of v_t on y_t :

$$\gamma = \frac{\text{Cov}(v_t, y_t)}{\text{Var}(y_t)}. \quad (6)$$

In this equilibrium, $y_t = I\chi^N(v_t - \bar{v}) + u_t$, hence:

$$\text{Cov}(v_t, y_t) = \text{Cov}(v_t, I\chi^N(v_t - \bar{v}) + u_t) = \text{Cov}(v_t, I\chi^N v_t - I\chi^N \bar{v}) = I\chi^N \text{Var}(v_t) = I\chi^N \sigma_v^2,$$

since u_t is independent of v_t and $\bar{v}$ is a constant. Note that $\chi^N > 0$ (speculators buy when $v_t > \bar{v}$ and sell when $v_t < \bar{v}$), therefore $\gamma > 0$, which in turn implies $\lambda > 0$, confirming our earlier assumption. Furthermore:

$$\frac{\partial^2 P}{\partial \chi_i^2} = (v_t - \bar{v})^2 (-2\lambda) < 0,$$

so the value we obtained for χ^N was indeed a maximum.

We now compute $\text{Var}(y_t)$:

$$\text{Var}(y_t) = \text{Var}(I\chi^N(v_t - \bar{v}) + u_t) = \text{Var}(I\chi^N v_t - I\chi^N \bar{v} + u_t) = \text{Var}(I\chi^N v_t + u_t) = I^2(\chi^N)^2 \sigma_v^2 + \sigma_u^2.$$

Consequently:

$$\gamma^N = \frac{I\chi^N \sigma_v^2}{I^2(\chi^N)^2 \sigma_v^2 + \sigma_u^2} = \frac{I\chi^N}{(I\chi^N)^2 + (\sigma_u/\sigma_v)^2}. \quad (7)$$

Finally, we derive the expected per-speculator profit. Substituting $p_t = \bar{v} + \lambda^N y_t$ and $x_{i,t} = \chi^N(v_t - \bar{v})$ into $\pi_{i,t} = (v_t - p_t)x_{i,t}$:

$$\pi_{i,t} = \chi^N(v_t - \bar{v})(v_t - \bar{v} - \lambda^N y_t).$$

Taking unconditional expectation and substituting $y_t = I\chi^N(v_t - \bar{v}) + u_t$:

$$\pi^N = \mathbb{E} [\chi^N(v_t - \bar{v})(v_t - \bar{v} - \lambda^N(I\chi^N(v_t - \bar{v}) + u_t))] = \chi^N(1 - \lambda^N I\chi^N) \mathbb{E}[(v_t - \bar{v})^2],$$

since $(v_t - \bar{v})$ is independent of u_t and $\mathbb{E}[u_t] = 0$. This gives:

$$\pi^N = (1 - \lambda^N I\chi^N) \chi^N \sigma_v^2. \quad (8)$$

3.3 Cartel Equilibrium

Now suppose all I speculators form a cartel that maximizes the total profit of the group, namely, they all coordinate on the same intensity χ^M that maximizes:

$$\sum_{i=1}^I \pi_{i,t} = (v_t - p_t) I \chi^M (v_t - \bar{v})$$

Substituting $p_t = \bar{v} + \lambda^M y_t$ and $y_t = I \chi^M (v_t - \bar{v}) + u_t$:

$$\sum_{i=1}^I \pi_{i,t} = I \chi^M (v_t - \bar{v})^2 - I \lambda^M \chi^M (v_t - \bar{v}) (I \chi^M (v_t - \bar{v}) + u_t)$$

Taking the conditional expectation on v_t :

$$\begin{aligned} Q := \mathbb{E} \left[\sum_{i=1}^I \pi_{i,t} \mid v_t \right] &= I \chi^M (v_t - \bar{v})^2 - I^2 \lambda^M (\chi^M)^2 (v_t - \bar{v})^2 - I \lambda^M \chi^M (v_t - \bar{v}) \mathbb{E}[u_t \mid v_t] = \\ &= I \chi^M (v_t - \bar{v})^2 - I^2 \lambda^M (\chi^M)^2 (v_t - \bar{v})^2 \end{aligned}$$

since u_t is independent of v_t and $\mathbb{E}[u_t] = 0$.

Taking the first order condition with respect to χ^M :

$$\frac{\partial Q}{\partial \chi^M} = (v_t - \bar{v})^2 [I - 2I^2 \lambda^M \chi^M] = 0 \iff$$

$$\chi^M = \frac{1}{2I\lambda^M}. \quad (9)$$

Moreover (where $\lambda^M > 0$ is verified below):

$$\frac{\partial^2 Q}{\partial (\chi^M)^2} = (v_t - \bar{v})^2 (-2I^2 \lambda^M) < 0,$$

so it is indeed a maximum.

From the definition of λ :

$$\lambda^M = \frac{\theta \gamma^M + \xi}{\theta + \xi^2}. \quad (10)$$

The derivations of γ^M and π^M follow analogously to the Nash case, replacing χ^N and λ^N with χ^M and λ^M . Note that $\lambda^M > 0$ for the same reason as λ^N . Consequently:

$$\gamma^M = \frac{I \chi^M}{(I \chi^M)^2 + (\sigma_u / \sigma_v)^2}, \quad (11)$$

and

$$\pi^M = (1 - \lambda^M I \chi^M) \chi^M \sigma_v^2. \quad (12)$$

3.4 Closed Forms and Numerical Results

When $\xi = 0$, the expression for λ^N simplifies to $\lambda^N = \gamma^N$. Substituting into the expression for χ^N and rearranging:

$$\chi^N = \frac{1}{(I+1)\gamma^N} = \frac{1}{I+1} \cdot \frac{(I\chi^N)^2 + (\sigma_u/\sigma_v)^2}{I\chi^N}.$$

Multiplying both sides by $(I+1)\chi^N$:

$$(I+1)I(\chi^N)^2 = I^2(\chi^N)^2 + \frac{\sigma_u^2}{\sigma_v^2} \implies (\chi^N)^2 = \frac{\sigma_u^2}{I\sigma_v^2},$$

and therefore:

$$\chi^N = \frac{\sigma_u}{\sqrt{I}\sigma_v}. \quad (13)$$

Substituting into $\lambda^N = \gamma^N = \frac{I\chi^N}{(I\chi^N)^2 + (\sigma_u/\sigma_v)^2}$:

$$\lambda^N = \frac{I \cdot \frac{\sigma_u}{\sqrt{I}\sigma_v}}{\left(\frac{I\sigma_u}{\sqrt{I}\sigma_v}\right)^2 + \left(\frac{\sigma_u}{\sigma_v}\right)^2} = \frac{\frac{\sqrt{I}\sigma_u}{\sigma_v}}{\frac{I\sigma_u^2}{\sigma_v^2} + \frac{\sigma_u^2}{\sigma_v^2}} = \frac{\frac{\sqrt{I}\sigma_u}{\sigma_v}}{\frac{(I+1)\sigma_u^2}{\sigma_v^2}} = \frac{\sqrt{I}\sigma_u}{\sigma_v} \cdot \frac{\sigma_v^2}{(I+1)\sigma_u^2},$$

and therefore:

$$\lambda^N = \frac{\sqrt{I}\sigma_v}{(I+1)\sigma_u}. \quad (14)$$

The cartel derivation is analogous, using $\chi^M = \frac{1}{2I\lambda^M}$ in place of $\chi^N = \frac{1}{(I+1)\lambda^N}$. Substituting $\lambda^M = \gamma^M$ at $\xi = 0$:

$$\chi^M = \frac{1}{2I} \cdot \frac{(I\chi^M)^2 + (\sigma_u/\sigma_v)^2}{I\chi^M}.$$

Multiplying both sides by $2I^2\chi^M$:

$$2I^2(\chi^M)^2 = I^2(\chi^M)^2 + \frac{\sigma_u^2}{\sigma_v^2} \implies I^2(\chi^M)^2 = \frac{\sigma_u^2}{\sigma_v^2},$$

and therefore:

$$\chi^M = \frac{\sigma_u}{I\sigma_v}. \quad (15)$$

Substituting into $\lambda^M = \gamma^M = \frac{I\chi^M}{(I\chi^M)^2 + (\sigma_u/\sigma_v)^2}$:

$$\lambda^M = \frac{I \cdot \frac{\sigma_u}{I\sigma_v}}{\left(\frac{\sigma_u}{\sigma_v}\right)^2 + \left(\frac{\sigma_u}{\sigma_v}\right)^2} = \frac{\frac{\sigma_u}{\sigma_v}}{2 \cdot \frac{\sigma_u^2}{\sigma_v^2}} = \frac{\sigma_v}{2\sigma_u},$$

and therefore:

$$\lambda^M = \frac{\sigma_v}{2\sigma_u}. \quad (16)$$

Now, since for $I \geq 2$, $\frac{1}{\sqrt{I}} > \frac{1}{I}$, we have $\chi^N > \chi^M$ if we compare (13) and (15).

Substituting the closed-form expressions into $\pi^N = (1 - \lambda^N I \chi^N) \chi^N \sigma_v^2$:

$$\pi^N = \left(1 - \frac{\sqrt{I} \sigma_v}{(I+1) \sigma_u} \cdot I \cdot \frac{\sigma_u}{\sqrt{I} \sigma_v}\right) \frac{\sigma_u}{\sqrt{I} \sigma_v} \sigma_v^2 = \left(1 - \frac{I}{I+1}\right) \frac{\sigma_u \sigma_v}{\sqrt{I}},$$

and therefore:

$$\pi^N = \frac{\sigma_u \sigma_v}{\sqrt{I}(I+1)}. \quad (17)$$

Similarly, substituting into $\pi^M = (1 - \lambda^M I \chi^M) \chi^M \sigma_v^2$:

$$\pi^M = \left(1 - \frac{\sigma_v}{2\sigma_u} \cdot I \cdot \frac{\sigma_u}{I\sigma_v}\right) \frac{\sigma_u}{I\sigma_v} \sigma_v^2 = \left(1 - \frac{1}{2}\right) \frac{\sigma_u \sigma_v}{I},$$

and therefore:

$$\pi^M = \frac{\sigma_u \sigma_v}{2I}. \quad (18)$$

To show $\pi^M > \pi^N$ for $I \geq 2$, we need:

$$\frac{\sigma_u \sigma_v}{2I} > \frac{\sigma_u \sigma_v}{\sqrt{I}(I+1)} \iff \sqrt{I}(I+1) > 2I \iff I^{3/2} + \sqrt{I} > 2I.$$

Defining $f(I) = I^{3/2} + \sqrt{I} - 2I$ and factoring out $\sqrt{I}$:

$$f(I) = \sqrt{I}(I + 1 - 2\sqrt{I}) = \sqrt{I}(\sqrt{I} - 1)^2.$$

Since $\sqrt{I} > 0$ and $(\sqrt{I} - 1)^2 \geq 0$ for all I , we have $f(I) \geq 0$, with equality only at $I = 1$. Therefore $f(I) > 0$ for all $I \geq 2$, which proves $\pi^M > \pi^N$.

When $\xi = 0$, information-insensitive investors are absent and the model reduces to the standard Kyle (1985) framework. In the Nash equilibrium, each speculator trades aggressively on their private signal, taking the strategies of others as given. However, since the market maker sets a price proportional to total order flow y_t , the collective aggressive trading drives prices closer to the fundamental value, reducing the profit margin $(v_t - p_t)$ for every speculator. The cartel internalizes this effect by coordinating on a lower trading intensity $\chi^M < \chi^N$, which keeps prices less informative and preserves larger profit margins. As a result, even though each cartel member trades less, the per-speculator profit is higher under the cartel than in the Nash equilibrium for $I \geq 2$, highlighting the incentive for speculators to collude.

To further illustrate these results numerically, a Python script was implemented to compute λ^N , χ^N , π^N and λ^M , χ^M , π^M for $\xi = 0$, $\sigma_v = 1$, $\sigma_u = 0.1$, and $I = 1, 2, \dots, 10$ using the closed-form expressions derived above. Figure 1 displays the resulting plots.

The plots show that λ^M is constant in I while λ^N decreases, reflecting the fact that competition among Nash speculators reduces price impact, whereas the cartel maintains a fixed level of market illiquidity regardless of group size. Both π^N and π^M decrease in I , as the per-speculator share of total profits diminishes with more participants. However, π^N falls strictly faster than π^M : in the Nash equilibrium, each speculator cannot coordinate to avoid the competition effect, so increasing I undermines individual profits through the price impact of rivals. The cartel

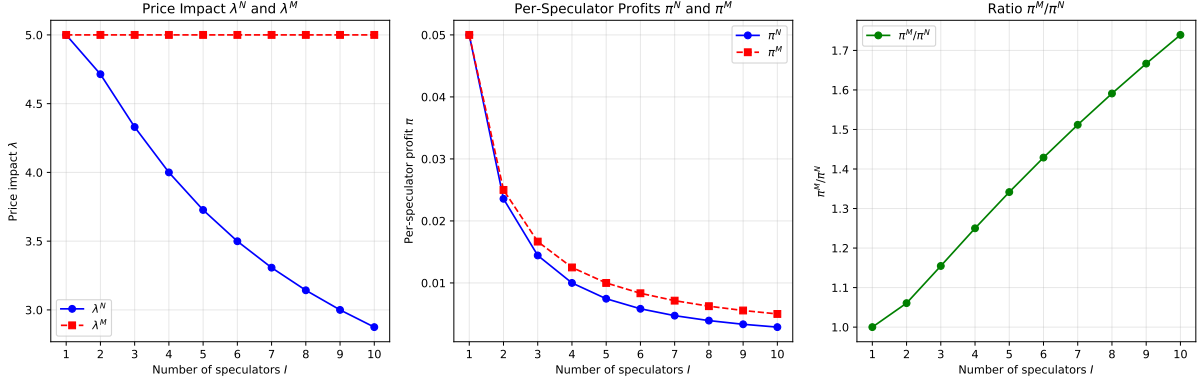


Figure 1: Price impact λ , per-speculator profits π , and profit ratio π^M/π^N under Nash and cartel equilibria, as functions of the number of speculators I , for $\xi = 0$, $\sigma_v = 1$, $\sigma_u = 0.1$.

internalizes this competition effect, limiting total order flow and preserving profit margins more effectively. As a result, the ratio π^M/π^N is strictly increasing in I , confirming that the incentive to collude grows with the number of speculators: the larger the group, the greater the gain from coordination relative to competitive trading.

4 Market Quality

Market quality refers to how well a financial market fulfills its main functions: incorporating information into prices, enabling trade at low cost, and limiting the exploitation of uninformed participants. In this model, noise traders represent liquidity traders, investors who trade for reasons unrelated to the fundamental value v_t , such as portfolio rebalancing or hedging needs. We measure market quality through three indicators:

- **Price informativeness:** $\mathcal{I} = \frac{1}{\text{Var}(v_t|p_t)} = \frac{I^2 \chi^2 \sigma_v^2 + \sigma_u^2}{\sigma_v^2 \sigma_u^2}$, which measures how accurately the market price reflects the fundamental value v_t . A higher $\mathcal{I}$ indicates that prices are more informative.
- **Market liquidity:** $\mathcal{L} = \frac{1}{\lambda}$, which measures how cheaply traders can execute orders without moving prices. A higher $\mathcal{L}$ indicates a more liquid market.
- **Noise-trader losses:** $W_N = \lambda \sigma_u^2$, which measures the average losses incurred by noise traders due to adverse selection.

For $\xi = 0$, we see from (14) that as I increases, λ^N decreases (strictly for $I \geq 2$), since if we define $f(I) = \frac{\sqrt{I}}{I+1}$:

$$f'(I) = \frac{\frac{1}{2\sqrt{I}}(I+1) - \sqrt{I}}{(I+1)^2} = \frac{-I+1}{2\sqrt{I}(I+1)^2} < 0 \quad \text{for } I \geq 2,$$

and $f'(1) = 0$. The price informativeness in the Nash equilibrium is obtained by substituting

$\chi^N = \frac{\sigma_u}{\sqrt{I}\sigma_v}$ into the expression for $\mathcal{I}$:

$$\mathcal{I}^N = \frac{I^2(\chi^N)^2\sigma_v^2 + \sigma_u^2}{\sigma_v^2\sigma_u^2} = \frac{I^2 \cdot \frac{\sigma_u^2}{I\sigma_v^2} \cdot \sigma_v^2 + \sigma_u^2}{\sigma_v^2\sigma_u^2} = \frac{I\sigma_u^2 + \sigma_u^2}{\sigma_v^2\sigma_u^2} = \frac{I+1}{\sigma_v^2},$$

which is strictly increasing in I for all $I \geq 1$. The noise-trader losses in the Nash equilibrium are

$$W_N^N = \lambda^N \sigma_u^2 = \frac{\sqrt{I}\sigma_v}{(I+1)\sigma_u} \cdot \sigma_u^2 = \sigma_u\sigma_v \cdot f(I),$$

which is decreasing in I (strictly for $I \geq 2$) by the same argument as for λ^N .

Substituting $\chi^M = \frac{\sigma_u}{I\sigma_v}$ into the same expressions gives

$$\mathcal{I}^M = \frac{2}{\sigma_v^2}, \quad W_N^M = \frac{\sigma_u\sigma_v}{2},$$

both constant in I . The cartel therefore locks in a fixed level of price opacity and noise-trader exploitation regardless of group size, while the Nash equilibrium delivers progressively higher liquidity, more informative prices, and lower noise-trader losses as I grows.

To illustrate these results numerically, a Python script computes λ^N , λ^M , $\mathcal{I}^N$, $\mathcal{I}^M$, W_N^N , and W_N^M for $\xi = 0$, $\sigma_v = 1$, $\sigma_u = 0.1$, and $I = 1, 2, \dots, 10$. Figure 2 displays the resulting plots.

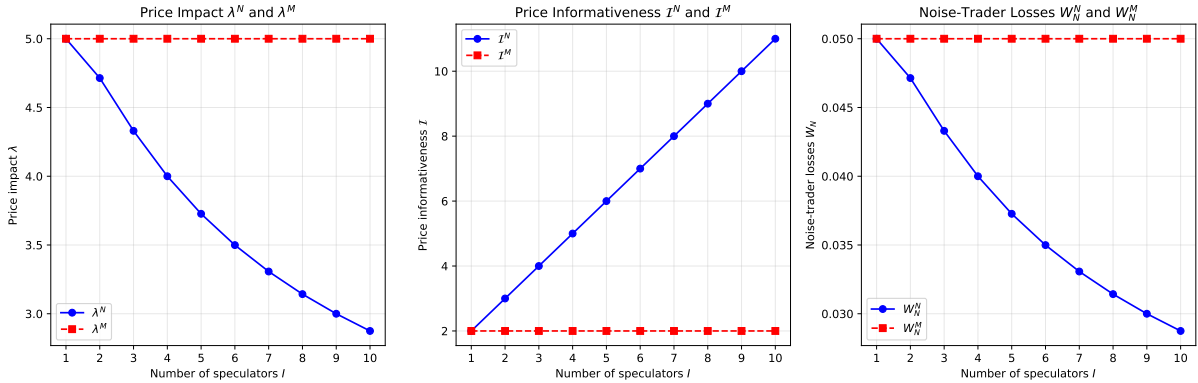


Figure 2: Price impact λ , price informativeness $\mathcal{I}$, and noise-trader losses W_N under Nash and cartel equilibria, as functions of the number of speculators I , for $\xi = 0$, $\sigma_v = 1$, $\sigma_u = 0.1$.

The economic interpretation of the figure is the following: the constancy of the cartel curves reflects that the aggregate response coefficient $I\chi^M = \sigma_u/\sigma_v$ is independent of group size: the cartel simply divides a fixed monopolist trading volume equally among its members. Competition, by contrast, drives each Nash speculator to trade more strongly on their signal, increasing the informational content of total order flow relative to noise. The result is a strict welfare ranking: for every $I \geq 2$, the Nash equilibrium delivers lower price impact, more informative prices, and smaller noise-trader losses than the cartel, and the gap widens as I grows. This confirms within the Dou et al. (2025) framework the result of Subrahmanyam (1991) that competition among risk-neutral informed traders monotonically improves market quality (Lemma 2(i)).

For $\xi > 0$, the equilibrium quantities χ^N , λ^N , χ^M , and λ^M cannot be solved in closed form and

must instead be obtained numerically as the solution to a fixed-point problem. Specifically, the Nash equilibrium conditions in equations (4), (5), and (7) can be reduced to finding the root of:

$$F(\chi) = \chi - \frac{1}{(I+1)\lambda(\gamma(\chi))}, \quad \text{where} \quad \gamma(\chi) = \frac{I\chi}{(I\chi)^2 + (\sigma_u/\sigma_v)^2}, \quad \lambda(\gamma) = \frac{\theta\gamma + \xi}{\theta + \xi^2},$$

and analogously for the cartel, replacing $(I+1)$ with $2I$. A Python script was implemented to find these roots using Brent's method, which is a robust bracketing algorithm that combines bisection, secant, and inverse quadratic interpolation. The bracketing interval $[a, b]$ is determined automatically by expanding the upper bound geometrically until a sign change is detected, which is necessary since for large ξ the equilibrium χ can be very large. The $\xi = 0$ case was used as a sanity check: the numerical results match the closed-form expressions derived in Section 3 exactly.

Table 1 reports the computed equilibrium quantities for $I = 2$, $\sigma_v = 1$, $\sigma_u = 0.1$, $\theta = 0.1$, and $\xi \in \{0, 50, 500\}$.

ξ	χ^N	λ^N	π^N	χ^M	λ^M	π^M
0	0.0707	4.7140	0.0236	0.0500	5.0000	0.0250
50	16.6663	0.0200	5.5554	12.4995	0.0200	6.2498
500	166.6666	0.0020	55.5555	125.0000	0.0020	62.5000

Table 1: Nash and cartel equilibrium quantities for $I = 2$, $\sigma_v = 1$, $\sigma_u = 0.1$, $\theta = 0.1$, and $\xi \in \{0, 50, 500\}$.

The results show that ξ has a dramatic effect on equilibrium behavior. As ξ increases, trading intensities χ^N and χ^M grow roughly in proportion to ξ , while price impact λ^N and λ^M go toward zero. The mechanism works as follows: large ξ means information-insensitive investors absorb most of the order flow, so the market maker's pricing rule becomes insensitive to informed trading, allowing speculators to trade aggressively at little cost. Profits consequently grow by several orders of magnitude relative to the $\xi = 0$ case. The absolute gap between π^M and π^N widens with ξ , indicating that information-insensitive investors amplify the gains from collusion. Nevertheless, the cartel always outperforms the Nash equilibrium ($\pi^M > \pi^N$) across all values of ξ , confirming that the incentive to collude persists regardless of the presence of information-insensitive investors.

Using the previous numerical solver, we compute λ^N , λ^M , $\mathcal{I}^N$, $\mathcal{I}^M$, W_N^N , and W_N^M for $\xi = 500$, $\theta = 0.1$, $\sigma_v = 1$, $\sigma_u = 0.1$, and $I = 1, 2, \dots, 10$. The results are displayed in Figure 3.

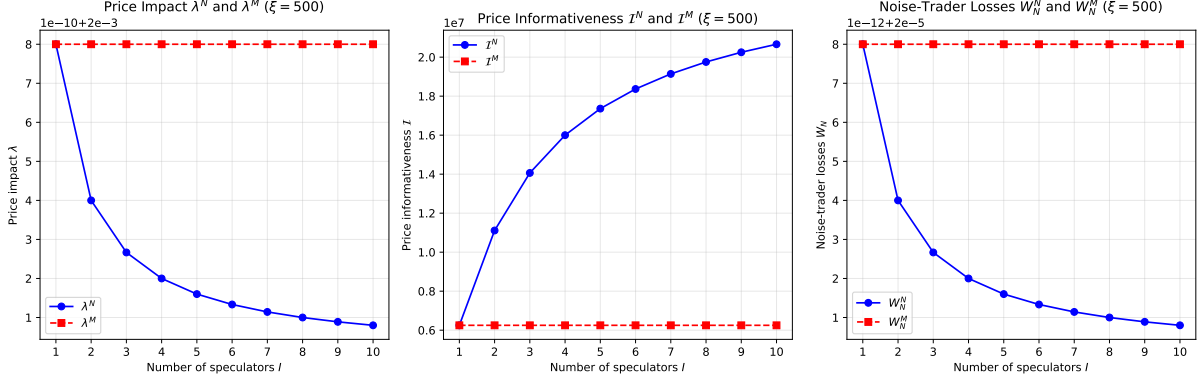


Figure 3: Price impact λ , price informativeness $\mathcal{I}$, and noise-trader losses W_N under Nash and cartel equilibria as functions of the number of speculators I , for $\xi = 500$, $\theta = 0.1$, $\sigma_v = 1$, $\sigma_u = 0.1$.

Both λ^N and λ^M are of order 0.002 at $\xi = 500$ versus order 5 at $\xi = 0$, a collapse of three orders of magnitude. The reason is that the pricing rule $\lambda = (\theta\gamma + \xi)/(\theta + \xi^2)$ is dominated by the ξ^2 term in the denominator for large ξ , giving $\lambda \approx 1/\xi = 0.002$. The market maker becomes insensitive to information and prices primarily off the liquidity parameter ξ , so insiders adjust χ but prices barely respond. The left panel of Figure 3 shows λ^N decreasing and λ^M flat, but the variations are of order 10^{-10} around a base of 0.002 , both equal 0.002000 to six decimal places for every I . The same mechanism flattens noise-trader losses $W_N = \lambda\sigma_u^2$, which sit at 2×10^{-5} with variations of order 10^{-12} .

The informativeness gap behaves differently. At $I = 10$, the ratio $\mathcal{I}^N/\mathcal{I}^M$ is roughly 3.3 at $\xi = 500$, against 5.5 at $\xi = 0$: information-insensitive investors close part of the gap because the cartel also trades more harshly in this regime. In absolute terms, both $\mathcal{I}^N$ and $\mathcal{I}^M$ are orders of magnitude larger at $\xi = 500$ than at $\xi = 0$, reflecting the much higher trading intensity of all speculators. The welfare ranking is the same: competition delivers substantially more informative prices than collusion, but the margin is smaller.

We now turn to a complementary way of quantifying the market impact of collusion: the effective number of insiders I^* , defined as the value such that $\lambda^N(I^*) = \lambda^M(I)$. That is, I^* is the number of Nash speculators needed to replicate the price impact of a cartel with I speculators. For $\xi = 0$, using the closed-form expressions for λ^N and λ^M :

$$\lambda^N(I^*) = \frac{\sqrt{I^*}\sigma_v}{(I^* + 1)\sigma_u}, \quad \lambda^M(I) = \frac{\sigma_v}{2\sigma_u}.$$

Setting these equal and canceling σ_v/σ_u (note that λ^M doesn't depend on I):

$$\frac{\sqrt{I^*}}{I^* + 1} = \frac{1}{2} \iff 2\sqrt{I^*} = I^* + 1 \iff 4I^* = (I^* + 1)^2 \iff I^{*2} - 2I^* + 1 = 0 \iff (I^* - 1)^2 = 0,$$

which has the unique solution $I^* = 1$, regardless of I , σ_v , and σ_u . This means that a cartel of I speculators generates exactly the same price impact as a Nash equilibrium with a single speculator, i.e. a monopolist. The cartel therefore trades as conservatively as a monopolist

regardless of its size, which makes intuitive sense: the cartel maximizes joint profits, and adding more members to the cartel does not change its aggregate trading behavior, it simply divides the same monopolist trading volume among more participants.

We now extend the analysis to $\xi = 500$, where closed-form expressions are no longer available. For each $I \in \{2, 3, 5\}$ we compute $\lambda^M(I)$ using the equilibrium solver and find I^* such that $\lambda^N(I^*) = \lambda^M(I)$ via Brent's method, treating I^* as the unknown. Uniqueness on the interval $[0.01, 10]$ is verified by a sign-change check on a 2,000-point grid.

For $\theta = 0.1$, $\sigma_v = 1$, $\sigma_u = 0.1$, the solver returns $I^* = 1$ in all three cases, with a unique solution each time. This is the same answer as at $\xi = 0$, and it is not an accident: the cartel FOC $\chi^M = 1/(2I\lambda^M)$ implies that the aggregate cartel coefficient $I\chi^M$ depends only on λ^M , which makes γ^M and hence λ^M constant in I . Combined with the identity $\lambda^N(1) = \lambda^M(1)$, this gives $\lambda^N(1) = \lambda^M(I)$ for every I , so $I^* = 1$ exactly.

Thus, the role of information-insensitive investors is not to change the fact that a cartel of any size trades like a monopolist; it is to compress the level of price impact at which it happens. At $\xi = 0$, $\lambda^N(1) = \lambda^M(I) = \sigma_v/(2\sigma_u) = 5$. At $\xi = 500$, the common value collapses to $\lambda \approx 1/\xi = 2 \times 10^{-3}$, three orders of magnitude smaller. Information-insensitive investors moderate the market impact of collusion (prices barely respond to informed order flow in either equilibrium) without altering the equivalence between the cartel and a monopolist.

5 Q-Learning and Algorithmic Collusion

5.1 Algorithm Description

We investigate whether model-free Q-learning algorithms, acting as informed speculators, can learn to collude without explicit communication. The simulation setup follows Dou et al. (2025): each speculator i maintains a Q-matrix $\hat{Q}_i(s, x)$ indexed by state $s = (p_{t-1}, v_{t-1}, v_t)$ and action x , chooses actions via ε -greedy exploration with state-dependent exploration rate $\varepsilon_t(v_k) = e^{-\beta t(v_k)}$, and updates its Q-matrix according to

$$\hat{Q}_{i,t+1}(s_t, x_{i,t}) = (1 - \alpha) \hat{Q}_{i,t}(s_t, x_{i,t}) + \alpha \left[\pi_{i,t} + \rho \max_{x'} \hat{Q}_{i,t}(s_{t+1}, x') \right], \quad (19)$$

with the max taken over the action grid corresponding to v_{t+1} . All other entries remain unchanged. We use $n_v = 10$ value grid points, state-dependent action grids of size $n_x = 15$ and price grids of size $n_p = 31$, with bracketing parameter $\iota = 0.1$. Lagged prices p_{t-1} are discretised on the price grid corresponding to v_{t-1} , since they were generated by actions chosen for that value state. The Q-matrix is initialised as

$$\hat{Q}_{i,0}(s, x_n) = \frac{1}{(1 - \rho) n_x} \sum_{x_{-i} \in \mathcal{X}_k} [v_k - (\bar{v} + \lambda^N(x_n + (I - 1)x_{-i}))] x_n, \quad (20)$$

which is the discounted payoff to speculator i if the other speculators randomise uniformly and the noise trader's order is zero, replicated across all (v_{t-1}, p_{t-1}) slices.

The degree of collusion is measured by

$$\Delta_\chi = \frac{\chi^N - \bar{\chi}^C}{\chi^N - \chi^M}, \quad (21)$$

where $\bar{\chi}^C$ is the average learned trading intensity: $\Delta_\chi = 0$ corresponds to Nash, $\Delta_\chi = 1$ to perfect cartel. Throughout, we use $\bar{v} = 1$, $\sigma_v = 1$, $\sigma_u = 0.1$, $I = 2$, $\rho = 0.95$, $\theta = 0.1$, $\alpha = 0.05$, $\beta = 5 \times 10^{-6}$. The discretisation grids depend on χ^N , χ^M , and λ^N and are recomputed for each value of ξ .

5.2 Simulation Results

As a baseline sanity check we ran $I = 1$, where $\chi^N = \chi^M = \sigma_u/\sigma_v = 0.1$ and the action grid collapses to a single point at every value-grid node ($x_k^N = x_k^M$). The simulation returns $\bar{\chi}^C = 0.100000$, matching the closed form exactly.

This should be interpreted as a consistency check rather than evidence of non-trivial strategic learning. When $I = 1$, the Nash and cartel order sizes are identical at every value-grid point, so the prescribed action-grid construction becomes degenerate: the interval $[x_k^M - \iota(x_k^N - x_k^M), x_k^N + \iota(x_k^N - x_k^M)]$ collapses to a single point when $x_k^N = x_k^M$, making every action on the grid identical. The test confirms that the discretization, state transitions, market-clearing step, and Q-learning updates are implemented coherently.

We now turn to the more interesting case of $I = 2$ informed speculators, where $\chi^N \neq \chi^M$ and the Q-learning algorithm must make a choice between the high Nash intensity and the conservative cartel intensity. The closed-form benchmarks at $\xi = 0$, $I = 2$ are $\chi^N = \sigma_u/(\sqrt{2}\sigma_v) \approx 0.0707$, $\chi^M = \sigma_u/(2\sigma_v) = 0.05$, and $\lambda^N = \sqrt{2}\sigma_v/(3\sigma_u) \approx 4.714$. The action grid is now non-degenerate, spanning trading intensities that allow speculators to deviate below the cartel level or above the Nash level by a factor of $\iota = 0.1$.

Implementation choice: three update protocols. The protocol can be read in more than one way: each speculator maintains a Q-matrix $\hat{Q}_i(s, x)$ (with subscript i), yet the speculators are also described as sharing the same Q-matrix and update rule. We compare three variants:

- **Separate matrices.** Each speculator $i \in \{1, 2\}$ maintains its own Q-matrix $\hat{Q}_i$, identically initialized via Eq. (20) and updated only by its own experience according to Eq. (19). Symmetry between agents emerges endogenously from identical dynamics rather than from enforcing a single shared matrix. This is the obvious interpretation and what the paper uses.
- **Shared matrix, sequential updates.** A single Q-matrix is consulted by both agents. Each period, both agents' updates are applied to that matrix in sequence. When both agents happen to pick the same (state, action) cell in the same period, the second update operates on the value already modified by the first, introducing a path-dependence.
- **Shared matrix, averaged updates.** A single Q-matrix is again used. Targets are computed per agent and grouped by the cell they would update. When multiple agents

hit the same cell, their targets are averaged before a single update is applied, making the update order-independent and symmetric across agents.

The separate-matrices variant is our primary specification; the two shared-matrix variants serve as a robustness check on whether the collusive outcome is an artifact of any particular update protocol.

We measure convergence of the learned greedy policy rather than of the Q-values, which fluctuate at the constant rate $\alpha = 0.05$. The original paper requires every cell’s greedy action to remain unchanged for 10^6 consecutive periods, a criterion calibrated for their ensemble of 1,000 sessions of length 2×10^7 to 5×10^{10} . This is too strict for our single-session simulations, so we adopt a softer per-cell variant: at each diagnostic checkpoint, we report the fraction of (s, i) cells whose greedy action has been unchanged for at least 10^6 consecutive periods. Combined with the trajectory of $\bar{\chi}^C$, this provides a quantitative summary of policy stationarity.

Our initial runs at $T = 50 \times 10^6$ training periods revealed that under all three protocols the greedy policy was still drifting: the $\bar{\chi}^C$ trajectory was visibly trending downward, and the fraction of cells stable for at least 10^6 consecutive periods sat between roughly 10% and 20%, indicating that most of the policy was still changing. To obtain interpretable estimates we extend the training horizon to $T = 500 \times 10^6$ periods.

Measurement of $\bar{\chi}^C$. For each variant we train the Q-learning algorithms for $T = 500 \times 10^6$ periods, freeze the Q-matrix or matrices, and evaluate the resulting greedy policy over an additional $T_{\text{eval}} = 10^5$ periods with exploration and learning both turned off. Following Dou et al. (2025, Internet Appendix, eq. (IA.4.4)), for each speculator i we run the unrestricted OLS regression

$$x_{i,t} = \chi_{i,0}^C + \chi_{i,1}^C v_t + \epsilon_t$$

over the T_{eval} frozen-policy sample, and report

$$\bar{\chi}^C = \frac{1}{I} \sum_{i=1}^I \hat{\chi}_{i,1}^C.$$

This slope-based estimator is numerically more stable than the per-period ratio $x_{i,t}/(v_t - \bar{v})$, which is sensitive to realisations of v_t close to $\bar{v}$, and it directly tests whether the learned policy takes the theoretically-motivated linear form in $(v_t - \bar{v})$.

Results. Table 2 summarises the three runs. All three protocols yield substantial collusion, with $\Delta_\chi \in [0.72, 0.92]$, and the ordering is $\Delta_\chi^{\text{separate}} < \Delta_\chi^{\text{averaged}} < \Delta_\chi^{\text{sequential}}$.

Method	$\bar{\chi}^C$	Δ_χ	Cells stable $\geq 10^6$ periods
Separate	0.0558	0.720	98.5%
Sequential	0.0517	0.919	98.1%
Averaged	0.0520	0.902	98.4%

Table 2: Learned trading intensity and convergence diagnostic under the three update protocols at $\xi = 0$, $I = 2$, $T = 500 \times 10^6$. The “cells stable” column reports the fraction of (s, i) cells whose greedy action has been unchanged for at least 10^6 consecutive periods at the end of training.

The $\bar{\chi}^C$ trajectories for all three methods stabilize during the final 250×10^6 periods (fluctuations on the order of 10^{-4}), so the OLS estimates are reliable. The per-cell stability metric reaches near 98%, with 1-2% of cells showing ongoing argmax oscillation due to near-tied Q-values across adjacent cells; this would likely be eliminated under Dou et al. (2025)’s smaller learning rate $\alpha = 0.01$, but does not affect our conclusions.

Across all three protocols, the algorithms learn to trade well below the Nash intensity and capture between approximately 0.7 and 0.9 of the distance toward the perfect cartel. This shows that the collusive outcome is not an artifact of any particular method, but a feature of Q-learning in this setting. The collusive outcome arises without any communication or shared information beyond the Q-learning update rule.

The separate matrices method results in a noticeably lower Δ_χ than the two shared matrix protocols (0.72 versus 0.90-0.92). Since the stability is essentially identical ($\sim 98\%$) and $\bar{\chi}^C$ has stabilized in all three cases, this gap reflects a genuine difference in the learned equilibrium rather than a convergence feature. A natural explanation is that shared protocols apply two updates per cell per period (either sequential or averaged), which effectively doubles the learning rate at cells the two agents both visit. Faster Q-value separation then causes the argmax to commit more to the action that first gets an advantage during the exploration phase. If that action is conservative, the shared protocols over-prune more effectively and land closer to the cartel.

Moreover, our results are based on single simulation sessions, whereas Dou et al. (2025) report ensemble averages across 1,000 independent sessions. Given the paper’s observation that individual sessions can converge to different equilibria due to randomness in exploration, our reported Δ_χ values should be interpreted as point estimates with non-trivial variance. A more comprehensive analysis would average across multiple independent runs per protocol.

We now consider the regime $\xi = 500$, in which a substantial mass of information-insensitive investors absorbs informed order flow at prices close to $\bar{v}$. Since closed-form expressions are no longer available, we solve the fixed-point system from Section 4 numerically using Brent’s method and obtain

$$\chi^N \approx 166.667, \quad \chi^M = 125.000, \quad \lambda^N \approx 2.000 \times 10^{-3},$$

which differ from the $\xi = 0$ benchmarks by roughly three orders of magnitude. The intuition is that information-insensitive investors provide an additional channel of liquidity beyond the noise trader, collapsing λ^N and allowing informed speculators to trade much more aggressively in absolute terms while preserving the per-unit margin $v_t - p_t$. The action and price grids are rebuilt with these new equilibrium values. We run each of the three update protocols for the horizon $T = 500 \times 10^6$ periods, and evaluate the frozen greedy policy over $T_{\text{eval}} = 10^5$ additional periods using the same OLS estimator $\bar{\chi}^C = I^{-1} \sum_i \hat{\chi}_{i,1}^C$ as before.

Results. Table 3 summarizes the three runs.

Method	$\bar{\chi}^C$	Δ_χ	Cells stable $\geq 10^6$ periods
Separate	153.80	0.309	96.5%
Sequential	130.60	0.866	98.1%
Averaged	130.65	0.865	97.5%

Table 3: Learned trading intensity and convergence diagnostic under the three update protocols at $\xi = 500$, $I = 2$, $T = 500 \times 10^6$. Closed-form benchmarks: $\chi^N \approx 166.67$, $\chi^M = 125.00$.

The $\bar{\chi}^C$ trajectories stabilise across all three protocols during the second half of training, and the per-cell stability plateaus near 96-98%, matching the patterns observed at $\xi = 0$. The two speculators in the separate-matrices run learn nearly identical policies, so the gap with the shared-Q protocols is again a genuine equilibrium effect.

Placing the two regimes side by side reveals a surprising pattern:

Method	$\Delta_\chi^{\xi=0}$	$\Delta_\chi^{\xi=500}$
Separate	0.720	0.309
Sequential	0.919	0.866
Averaged	0.902	0.865

Table 4: Collusion index across regimes for the three update protocols.

The two shared-Q protocols deliver similar collusion in both regimes ($\Delta_\chi \approx 0.87$ - 0.92), while the separate-matrices protocol drops sharply from 0.72 at $\xi = 0$ to 0.31 at $\xi = 500$. To verify that this gap is not a session-level artifact, we re-ran the separate-matrices protocol at $\xi = 500$ with four additional random seeds. The five sessions returned Δ_χ values of $\{0.307, 0.306, 0.328, 0.315, 0.320\}$, with mean 0.316 and standard deviation 0.009, giving a 95% confidence interval of $[0.308, 0.323]$. The per-cell stability metric exceeded 92% in every run. The collusion gap between separate-matrices and shared-Q is therefore a genuine equilibrium difference between the protocols, not a sampling artifact. The IRF analysis in the next section disentangles the mechanisms underlying both regimes.

5.3 Impulse Response Analysis

The previous section showed that Q-learning algorithms converge to partially collusive strategies ($\Delta_\chi > 0$) in both the $\xi = 0$ and $\xi = 500$ regimes. We now ask what sustains this collusion. The authors identify two distinct mechanisms. The first, *price-trigger strategies*, works through implicit coordination: speculators monitor lagged prices and revert to aggressive trading whenever a price movement looks like a deviation by a rival. The second, *over-pruning bias*, works through learning: aggressive actions are punished more heavily by adverse noise-trading shocks during training, so the algorithms systematically undervalue and prune them, ending up with a conservative policy regardless of any rival’s behavior. Both mechanisms produce similar levels of steady-state collusion, but their signatures are different: price-trigger collusion reacts to an unexpected price movement with a punishment phase, while over-pruning collusion does not react at all.

To distinguish between the two mechanisms empirically, we build a noise shock impulse response function (IRF) following the protocol of Dou et al. (2025, Section 5.3). After the Q-matrices have stabilized, we freeze them and draw $N_{\text{sim}} = 1,000$ independent paths of fundamental values $\{v_1, \dots, v_H\}$ and noise realizations $\{u_1, \dots, u_H\}$ with horizon $H = 9$. Each path is simulated twice under common random numbers: once as a baseline, and once with an exogenous shock applied to the noise trader’s order at $t = 3$, $u_3 \leftarrow u_3 + u_{\text{shock}} \cdot \text{sgn}(v_3 - \bar{v})$, directed in line with the informed speculators’ trade. We use three shock magnitudes calibrated as 0.5%, 2.5%, and 7% of the average absolute order size $\mathbb{E}[|x_i|]$ under the learned policy, producing small, medium, and large price deviations at $t = 3$.

To aggregate responses across paths with different signs of $v_t - \bar{v}$, we define the sign-adjusted variables $\tilde{p}_t = (p_t - \bar{v})\text{sgn}(v_t - \bar{v})$ and $\tilde{x}_{i,t} = x_{i,t}\text{sgn}(v_t - \bar{v})$, which by construction are positive in expectation. For each variable we average across the N_{sim} paths and report the percentage deviation of the shocked average from the baseline (long-run) mean for price $\tilde{p}_t$, per-speculator profit π_t , and order flow $\tilde{x}_t$, at each horizon $t = 1, \dots, 9$.

The two mechanisms predict very different IRF signatures. Under price-trigger, an elevated p_{t-1} is read as a sign of defection from cartel behavior, so both speculators trade at $t = 4$ to punish the suspected deviator. If the trigger threshold is crossed, this generates an order-flow response and an amplified price deviation at $t = 4$ that can match or exceed the $t = 3$ spike (Dou et al., 2025, Figure 3). Under over-pruning, the learned policy is essentially state-independent, so the shocked p_3 leaves the $t = 4$ actions unchanged and prices go back to baseline immediately (Dou et al., 2025, Figure 4).

We begin with $\xi = 0$. Across all three update protocols and all three shock sizes, the IRFs show the same pattern. The price spikes at $t = 3$ in proportion to the shock magnitude ($\approx 0.26\%$, 1.31% , and 3.66% for the small, medium, and large shocks respectively), and the per-speculator profit drops by approximately the same percentage in absolute terms, reflecting the price-impact response $\Delta p_3 \approx \lambda^N u_{\text{shock}}$. At $t = 4$ the price deviation is dampened in every case, falling within $[-0.33\%, 0.20\%]$ across the three methods and three shock sizes: between one and two orders of magnitude smaller than the $t = 3$ spike, and with no consistent sign across methods. The

order-flow deviation at $t = 4$ stays within the same range. By $t = 6-7$ all responses have decayed to noise-floor levels.

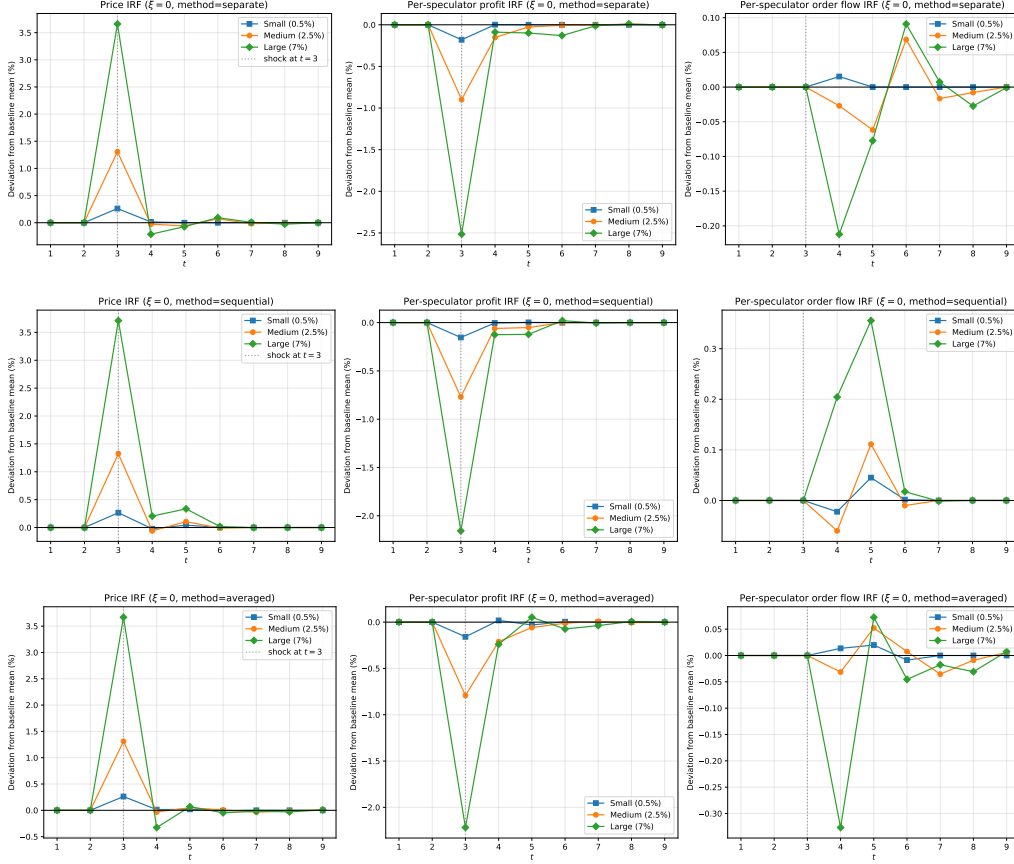


Figure 4: Impulse response functions at $\xi = 0$ for the three update methods (top to bottom: separate, sequential, averaged). Across all panels, the response decays to zero immediately after the $t = 3$ spike.

This pattern matches the over-pruning prediction: no order-flow adjustment at $t = 4$ and dampening of price deviation. The absence of any state-dependent response is exactly what the authors predict for case (iii) of their taxonomy (low ξ , low σ_u), where price-trigger strategies have no profitable foundation and over-pruning is the only mechanism available (Dou et al., 2025, Section 5.5).

The picture at $\xi = 500$ is somewhat different: the $t = 3$ spike is similar in proportional terms to the $\xi = 0$ case (0.25%, 1.23%, 3.44% across all methods), but the response at $t = 4$ and beyond depends strongly on the update protocol. Under the two shared protocols, both price and order flow exhibit large positive deviations at $t = 4$, often exceeding the spike at $t = 3$. The sequential medium shock, for example, yields $\tilde{p}_3 = 1.23\%$ followed by $\tilde{p}_4 = 4.40\%$, an amplification factor of 3.6. The $t = 4$ deviations approach a similar level ($\approx 4-5\%$) for medium and large shocks, while remaining $\approx 1-2\%$ for the small shock.

This is the saturation pattern Dou et al. (2025, Figure 3) identify as the evidence of a price-trigger punishment scheme: once the trigger threshold is crossed, both speculators revert to a coordinated punishment whose intensity is set by the trigger response itself rather than by the

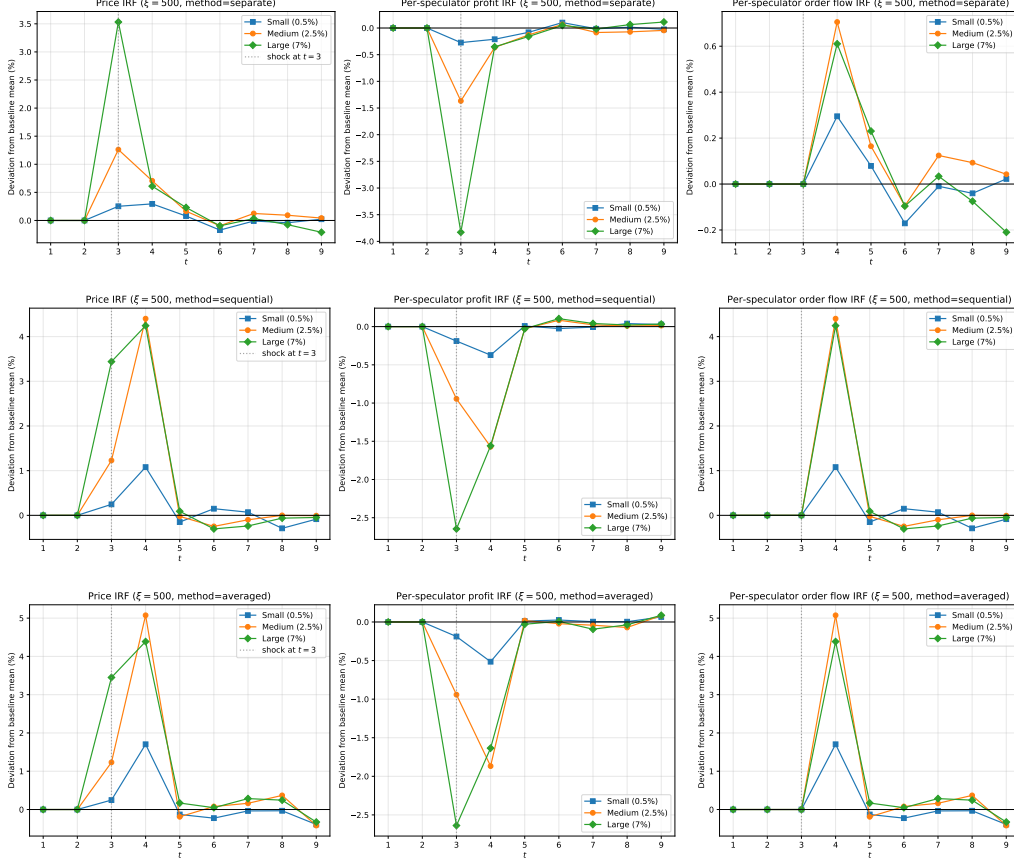


Figure 5: Impulse response functions at $\xi = 500$ for the three update methods (top to bottom: separate, sequential, averaged). The shared-Q protocols (middle, bottom) show amplification at $t = 4$ which is consistent with the price-trigger mechanism. The separate matrices method (top) shows a weaker but similar pattern.

magnitude of the original deviation. An elevated p_{t-1} is read as a sign of defection, and the speculators respond by trading more aggressively at $t = 4$ to punish the suspected deviator.

The separate-matrices protocol shows a weaker but similar pattern. The small shock generates essentially no response at $t = 4$ ($\tilde{p}_4 = -0.05\%$), while the medium and large shocks produce small but positive amplifications ($\tilde{p}_4 = 0.21\%$ and 0.13%). The threshold and saturation structure is also preserved, and the saturation level is much lower than under the shared-Q protocols. This matches the small-deviation case of Dou et al. (2025), where shocks below the threshold fail to trigger a response and the system reverts to baseline at $t = 4$. The lower saturation level under separate matrices is consistent with its lower collusion index ($\Delta_\chi^{\text{sep}} = 0.31$ versus $\Delta_\chi^{\text{shared}} \approx 0.87$): less collusion implies a less strict punishment.

All three protocols at $\xi = 500$ are therefore consistent with price-trigger collusion. The shared matrix methods deliver a sharper signature than separate matrices, possibly because common updates accelerate convergence toward conservative actions; the multi-seed evidence in Section 5.2 confirms this gap is a genuine protocol effect rather than session-level noise. The separate-matrices result, which follows the original paper more closely, confirms that the mechanism is present at $\xi = 500$ even at lower intensity.

6 Conclusion

The theoretical analysis established the benchmarks: at $\xi = 0$, the cartel intensity lies below the Nash intensity for $I \geq 2$, per-speculator profits satisfy $\pi^M > \pi^N$, and all three market-quality dimensions favor the competitive outcome, with the gap widening in I , in line with Subrahmanyam (1991). For $\xi > 0$, λ collapses toward $1/\xi$ and quantities rescale by orders of magnitude, but the rankings $\chi^N > \chi^M$ and $\pi^M > \pi^N$ are preserved.

The simulations show the collusive outcome emerges in practice across all three protocols and both regimes, with Δ_χ between roughly 0.3 and 0.9. The IRF analysis then identifies the mechanism in each regime: at $\xi = 0$, no $t = 4$ adjustment (the over-pruning signature); at $\xi = 500$, the threshold and saturation pattern of price-trigger collusion. Despite reaching comparable steady-state collusion levels, the two regimes produce qualitatively different IRF behavior, validating the two-mechanism theory of the authors within our setting. The magnitude is sensitive to the update protocol: separate matrices yields lower Δ_χ than a shared Q-matrix, particularly at $\xi = 500$; and a fuller treatment would estimate session-level distributions across all configurations, as Dou et al. (2025) do. However, the main result is clear: algorithms with no shared information, no communication, and no built-in incentive to cooperate learn to trade well below the competitive intensity, which suggests that the absence of explicit coordination is not a sufficient safeguard against algorithmic collusion.

References

- Calvano, E., Calzolari, G., Denicolò, V. and Pastorello, S. (2020). Artificial intelligence, algorithmic pricing, and collusion. *American Economic Review*, 110(10), 3267–3297.
- Dou, W. W., Goldstein, I. and Ji, Y. (2025). AI-powered trading, algorithmic collusion, and price efficiency. *NBER Working Paper 34054*.
- Kyle, A. S. (1985). Continuous auctions and insider trading. *Econometrica*, 53(6), 1315–1335.
- Subrahmanyam, A. (1991). Risk aversion, market liquidity, and price efficiency. *Review of Financial Studies*, 4(3), 417–441.

A Code

This appendix reproduces the Python code used to generate every numerical and graphical result in the report, together with the textual outputs of each cell. Plot outputs are omitted here as they appear inline in the main text. The code is organized by sections and follows the structure of the accompanying Jupyter notebook.

A.1 Closed-form Nash and cartel benchmarks

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Parameters
5 sigma_v = 1.0
6 sigma_u = 0.1
7 I_values = np.arange(1, 11)
8
9 # Closed-form expressions for  $\pi_i=0$ 
10 chi_N = sigma_u / (np.sqrt(I_values) * sigma_v)
11 chi_M = sigma_u / (I_values * sigma_v)
12 lambda_N = np.sqrt(I_values) * sigma_v / ((I_values + 1) * sigma_u)
13 lambda_M = sigma_v / (2 * sigma_u) * np.ones_like(I_values, dtype=float)
14 pi_N = sigma_u * sigma_v / (np.sqrt(I_values) * (I_values + 1))
15 pi_M = sigma_u * sigma_v / (2 * I_values)
16 ratio = pi_M / pi_N
17
18 # Plotting
19 fig, axes = plt.subplots(1, 3, figsize=(15, 5))
20
21 # Plot 1:  $\lambda^N$  and  $\lambda^M$ 
22 axes[0].plot(I_values, lambda_N, 'b-o', label=r'$\lambda^N$')
23 axes[0].plot(I_values, lambda_M, 'r--s', label=r'$\lambda^M$')
24 axes[0].set_xlabel(r'Number of speculators  $I$ ')
25 axes[0].set_ylabel(r'Price impact  $\lambda$ ')
26 axes[0].set_title(r'Price Impact  $\lambda^N$  and  $\lambda^M$ ')
27 axes[0].legend()
28 axes[0].grid(True, alpha=0.3)
29 axes[0].set_xticks(I_values)
30
31 # Plot 2:  $\pi^N$  and  $\pi^M$ 
32 axes[1].plot(I_values, pi_N, 'b-o', label=r'$\pi^N$')
33 axes[1].plot(I_values, pi_M, 'r--s', label=r'$\pi^M$')
34 axes[1].set_xlabel(r'Number of speculators  $I$ ')
35 axes[1].set_ylabel(r'Per-speculator profit  $\pi$ ')
36 axes[1].set_title(r'Per-Speculator Profits  $\pi^N$  and  $\pi^M$ ')
37 axes[1].legend()
38 axes[1].grid(True, alpha=0.3)
39 axes[1].set_xticks(I_values)
40
41 # Plot 3: ratio  $\pi^M/\pi^N$ 
42 axes[2].plot(I_values, ratio, 'g-o', label=r'$\pi^M/\pi^N$')
43 axes[2].set_xlabel(r'Number of speculators  $I$ ')
```

```

44 axes[2].set_ylabel(r'$\pi^M/\pi^N$')
45 axes[2].set_title(r'Ratio $\pi^M/\pi^N$')
46 axes[2].legend()
47 axes[2].grid(True, alpha=0.3)
48 axes[2].set_xticks(I_values)
49
50 plt.tight_layout()
51 plt.savefig('lambda_pi_ratio_plots.pdf', bbox_inches='tight')
52 plt.savefig('lambda_pi_ratio_plots.png', bbox_inches='tight', dpi=150)
53 plt.close()
54 print("Plots saved successfully.")
55
56 # Print table of values
57 print(f"\n{'I':>4} {'chi_N':>10} {'chi_M':>10} {'lambda_N':>10} "
58       f"{'lambda_M':>10} {'pi_N':>10} {'pi_M':>10} {'pi_M/pi_N':>10}")
59 print("-" * 80)
60 for i, I in enumerate(I_values):
61     print(f"{'I':>4} {'chi_N[i]':>10.4f} {'chi_M[i]':>10.4f} {'lambda_N[i]':>10.4f} "
62           f"{'lambda_M[i]':>10.4f} {'pi_N[i]':>10.4f} {'pi_M[i]':>10.4f} {'ratio[i "
63         ↪ ":>10.4f}"]")

```

Output:

Plots saved successfully.

I	chi_N ↪ pi_N	chi_M	lambda_N	lambda_M	pi_N	pi_M	pi_M/

↪							
1	0.1000	0.1000	5.0000	5.0000	0.0500	0.0500	
↪	1.0000						
2	0.0707	0.0500	4.7140	5.0000	0.0236	0.0250	
↪	1.0607						
3	0.0577	0.0333	4.3301	5.0000	0.0144	0.0167	
↪	1.1547						
4	0.0500	0.0250	4.0000	5.0000	0.0100	0.0125	
↪	1.2500						
5	0.0447	0.0200	3.7268	5.0000	0.0075	0.0100	
↪	1.3416						
6	0.0408	0.0167	3.4993	5.0000	0.0058	0.0083	
↪	1.4289						
7	0.0378	0.0143	3.3072	5.0000	0.0047	0.0071	
↪	1.5119						
8	0.0354	0.0125	3.1427	5.0000	0.0039	0.0063	
↪	1.5910						
9	0.0333	0.0111	3.0000	5.0000	0.0033	0.0056	
↪	1.6667						
10	0.0316	0.0100	2.8748	5.0000	0.0029	0.0050	
↪	1.7393						

A.2 Market quality at $\xi = 0$

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Parameters
5 sigma_v = 1.0
6 sigma_u = 0.1
7 I_values = np.arange(1, 11)
8
9 # Closed-form expressions ( $\xi = 0$ )
10 # Nash equilibrium
11 chi_N = sigma_u / (np.sqrt(I_values) * sigma_v)
12 lambda_N = np.sqrt(I_values) * sigma_v / ((I_values + 1) * sigma_u)
13
14 # Cartel equilibrium
15 chi_M = sigma_u / (I_values * sigma_v)
16 lambda_M = sigma_v / (2 * sigma_u) * np.ones_like(I_values, dtype=float)
17
18 # Market quality measures
19 # Price informativeness:  $I = (I^2 * \chi^2 * \sigma_v^2 + \sigma_u^2) / (\sigma_v^2 * \sigma_u^2)$ 
20 I_N = (I_values**2 * chi_N**2 * sigma_v**2 + sigma_u**2) / (sigma_v**2 *
21     ↪ sigma_u**2)
22 I_M = (I_values**2 * chi_M**2 * sigma_v**2 + sigma_u**2) / (sigma_v**2 *
23     ↪ sigma_u**2)
24
25 # Noise-trader losses:  $W_N = \lambda * \sigma_u^2$ 
26 W_N_nash = lambda_N * sigma_u**2
27 W_N_cartel = lambda_M * sigma_u**2
28
29 # Plotting
30 fig, axes = plt.subplots(1, 3, figsize=(15, 5))
31
32 # Plot 1:  $\lambda^N$  and  $\lambda^M$ 
33 axes[0].plot(I_values, lambda_N, 'b-o', label=r'$\lambda^N$')
34 axes[0].plot(I_values, lambda_M, 'r--s', label=r'$\lambda^M$')
35 axes[0].set_xlabel(r'Number of speculators  $I$ ')
36 axes[0].set_ylabel(r'Price impact  $\lambda$ ')
37 axes[0].set_title(r'Price Impact  $\lambda^N$  and  $\lambda^M$ ')
38 axes[0].legend()
39 axes[0].grid(True, alpha=0.3)
40 axes[0].set_xticks(I_values)
41
42 # Plot 2:  $I^N$  and  $I^M$ 
43 axes[1].plot(I_values, I_N, 'b-o', label=r'$\mathcal{I}^N$')
44 axes[1].plot(I_values, I_M, 'r--s', label=r'$\mathcal{I}^M$')
45 axes[1].set_xlabel(r'Number of speculators  $I$ ')
46 axes[1].set_ylabel(r'Price informativeness  $\mathcal{I}$ ')
47 axes[1].set_title(r'Price Informativeness  $\mathcal{I}^N$  and  $\mathcal{I}^M$ ')
48 axes[1].legend()
49 axes[1].grid(True, alpha=0.3)
50 axes[1].set_xticks(I_values)

```



```

49
50 # Plot 3:  $W_N^N$  and  $W_N^M$ 
51 axes[2].plot(I_values, W_N_nash, 'b-o', label=r'$W_N^N$')
52 axes[2].plot(I_values, W_N_cartel, 'r--s', label=r'$W_N^M$')
53 axes[2].set_xlabel(r'Number of speculators $I$')
54 axes[2].set_ylabel(r'Noise-trader losses $W_N$')
55 axes[2].set_title(r'Noise-Trader Losses $W_N^N$ and $W_N^M$')
56 axes[2].legend()
57 axes[2].grid(True, alpha=0.3)
58 axes[2].set_xticks(I_values)
59
60 plt.tight_layout()
61 plt.savefig('market_quality_plots.pdf', bbox_inches='tight')
62 plt.close()
63
64 print("Plots saved successfully.")
65
66 # Print numerical values for verification
67 print("\nI | lambda_N | lambda_M | I_N      | I_M      | W_N_N    | W_N_M")
68 print("-" * 70)
69 for i, I in enumerate(I_values):
70     print(f"{I:2d} | {lambda_N[i]:.4f} | {lambda_M[i]:.4f} | {I_N[i]:.4f} | {I_M[i]:.4f} | {W_N_nash[i]:.4f} | {W_N_cartel[i]:.4f}")

```

Output:

Plots saved successfully.

I	lambda_N	lambda_M	I_N	I_M	W_N_N	W_N_M
1	5.0000	5.0000	2.0000	2.0000	0.0500	0.0500
2	4.7140	5.0000	3.0000	2.0000	0.0471	0.0500
3	4.3301	5.0000	4.0000	2.0000	0.0433	0.0500
4	4.0000	5.0000	5.0000	2.0000	0.0400	0.0500
5	3.7268	5.0000	6.0000	2.0000	0.0373	0.0500
6	3.4993	5.0000	7.0000	2.0000	0.0350	0.0500
7	3.3072	5.0000	8.0000	2.0000	0.0331	0.0500
8	3.1427	5.0000	9.0000	2.0000	0.0314	0.0500
9	3.0000	5.0000	10.0000	2.0000	0.0300	0.0500
10	2.8748	5.0000	11.0000	2.0000	0.0287	0.0500

A.3 Numerical equilibrium solver

```

1 import numpy as np
2 from scipy.optimize import brentq
3
4 # Parameters
5 sigma_v = 1.0
6 sigma_u = 0.1
7 theta = 0.1
8 I = 2
9 xi_values = [0, 50, 500]

```

```

10
11 def gamma_func(chi, I, sigma_v, sigma_u):
12     """Regression coefficient gamma given chi."""
13     return (I * chi) / ((I * chi)**2 + (sigma_u / sigma_v)**2)
14
15 def lambda_func(gamma, xi, theta):
16     """Price impact lambda given gamma."""
17     return (theta * gamma + xi) / (theta + xi**2)
18
19 def nash_fixed_point(chi, I, sigma_v, sigma_u, xi, theta):
20     """Root gives Nash equilibrium chi^N."""
21     g = gamma_func(chi, I, sigma_v, sigma_u)
22     lam = lambda_func(g, xi, theta)
23     return chi - 1.0 / ((I + 1) * lam)
24
25 def cartel_fixed_point(chi, I, sigma_v, sigma_u, xi, theta):
26     """Root gives cartel equilibrium chi^M."""
27     g = gamma_func(chi, I, sigma_v, sigma_u)
28     lam = lambda_func(g, xi, theta)
29     return chi - 1.0 / (2 * I * lam)
30
31 def find_bracket(f, args, lower=1e-10, start=10.0, factor=10.0, max_iter=20):
32     """Automatically expand upper bound until bracket found."""
33     upper = start
34     for _ in range(max_iter):
35         if f(lower, *args) * f(upper, *args) < 0:
36             return lower, upper
37         upper *= factor
38     raise ValueError(f"Could not find bracket after {max_iter} iterations.")
39
40 def solve_equilibrium(I, sigma_v, sigma_u, xi, theta):
41     """Solve for Nash and cartel equilibria using Brent's method."""
42     args = (I, sigma_v, sigma_u, xi, theta)
43
44     # Nash
45     a, b = find_bracket(nash_fixed_point, args)
46     chi_N = brentq(nash_fixed_point, a, b, args=args)
47     gamma_N = gamma_func(chi_N, I, sigma_v, sigma_u)
48     lambda_N = lambda_func(gamma_N, xi, theta)
49     pi_N = (1 - lambda_N * I * chi_N) * chi_N * sigma_v**2
50
51     # Cartel
52     a, b = find_bracket(cartel_fixed_point, args)
53     chi_M = brentq(cartel_fixed_point, a, b, args=args)
54     gamma_M = gamma_func(chi_M, I, sigma_v, sigma_u)
55     lambda_M = lambda_func(gamma_M, xi, theta)
56     pi_M = (1 - lambda_M * I * chi_M) * chi_M * sigma_v**2
57
58     return chi_N, lambda_N, pi_N, chi_M, lambda_M, pi_M
59
60 # --- Compute and display results ---
61 print("=" * 80)

```

```

62 print(f"Parameters: I={I}, sigma_v={sigma_v}, sigma_u={sigma_u}, theta={theta}"
    ↪ )
63 print("=" * 80)
64 print(f"{'xi':>6} | {'chi_N':>10} | {'lambda_N':>10} | {'pi_N':>10} | "
65       f"{'chi_M':>10} | {'lambda_M':>10} | {'pi_M':>10}")
66 print("-" * 80)
67
68 results = {}
69 for xi in xi_values:
70     chi_N, lambda_N, pi_N, chi_M, lambda_M, pi_M = solve_equilibrium(
71         I, sigma_v, sigma_u, xi, theta)
72     results[xi] = (chi_N, lambda_N, pi_N, chi_M, lambda_M, pi_M)
73     print(f"{'xi':>6} | {'chi_N':>10.6f} | {'lambda_N':>10.6f} | {'pi_N':>10.6f} | "
74           f"{'chi_M':>10.6f} | {'lambda_M':>10.6f} | {'pi_M':>10.6f}")
75
76 print("=" * 80)
77
78 # --- Verify xi=0 against closed forms ---
79 print("\nVerification against closed forms for xi=0:")
80 chi_N_cf = sigma_u / (np.sqrt(I) * sigma_v)
81 lambda_N_cf = np.sqrt(I) * sigma_v / ((I + 1) * sigma_u)
82 pi_N_cf = sigma_u * sigma_v / (np.sqrt(I) * (I + 1))
83 chi_M_cf = sigma_u / (I * sigma_v)
84 lambda_M_cf = sigma_v / (2 * sigma_u)
85 pi_M_cf = sigma_u * sigma_v / (2 * I)
86
87 print(f" chi_N:    numerical={results[0][0]:.6f}, closed-form={chi_N_cf:.6f}")
88 print(f" lambda_N: numerical={results[0][1]:.6f}, closed-form={lambda_N_cf:.6f}
    ↪ }")
89 print(f" pi_N:    numerical={results[0][2]:.6f}, closed-form={pi_N_cf:.6f}")
90 print(f" chi_M:    numerical={results[0][3]:.6f}, closed-form={chi_M_cf:.6f}")
91 print(f" lambda_M: numerical={results[0][4]:.6f}, closed-form={lambda_M_cf:.6f}
    ↪ }")
92 print(f" pi_M:    numerical={results[0][5]:.6f}, closed-form={pi_M_cf:.6f}")

```

Output:

```

=====
    ↪
Parameters: I=2, sigma_v=1.0, sigma_u=0.1, theta=0.1
=====
    ↪
    xi |      chi_N |    lambda_N |      pi_N |      chi_M |    lambda_M |
    ↪      pi_M
-----
    ↪
    0 |    0.070711 |    4.714045 |    0.023570 |    0.050000 |    5.000000 |
    ↪      0.025000
    50 |    16.666333 |    0.020000 |    5.555444 |    12.499500 |    0.020001 |
    ↪      6.249750
    500 |   166.666633 |    0.002000 |   55.555544 |   124.999950 |    0.002000 |
    ↪      62.499975

```

```
=====
↪

Verification against closed forms for xi=0:
chi_N:    numerical=0.070711, closed-form=0.070711
lambda_N: numerical=4.714045, closed-form=4.714045
pi_N:     numerical=0.023570, closed-form=0.023570
chi_M:     numerical=0.050000, closed-form=0.050000
lambda_M: numerical=5.000000, closed-form=5.000000
pi_M:     numerical=0.025000, closed-form=0.025000
```

A.4 Comparative statics at $\xi = 500$

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Parameters specific to this experiment
5 xi = 500
6 I_values = np.arange(1, 11)
7
8 # --- Compute for all I ---
9 chi_N_arr = np.zeros(len(I_values))
10 lambda_N_arr = np.zeros(len(I_values))
11 pi_N_arr = np.zeros(len(I_values))
12 chi_M_arr = np.zeros(len(I_values))
13 lambda_M_arr = np.zeros(len(I_values))
14 pi_M_arr = np.zeros(len(I_values))
15
16 for idx, I in enumerate(I_values):
17     chi_N, lambda_N, pi_N, chi_M, lambda_M, pi_M = solve_equilibrium(
18         I, sigma_v, sigma_u, xi, theta
19     )
20     chi_N_arr[idx] = chi_N
21     lambda_N_arr[idx] = lambda_N
22     pi_N_arr[idx] = pi_N
23     chi_M_arr[idx] = chi_M
24     lambda_M_arr[idx] = lambda_M
25     pi_M_arr[idx] = pi_M
26
27 # --- Market quality measures ---
28 I_info_N = (I_values**2 * chi_N_arr**2 * sigma_v**2 + sigma_u**2) / (sigma_v**2
29     ↪ * sigma_u**2)
30 I_info_M = (I_values**2 * chi_M_arr**2 * sigma_v**2 + sigma_u**2) / (sigma_v**2
31     ↪ * sigma_u**2)
32 W_N_nash = lambda_N_arr * sigma_u**2
33 W_N_cartel = lambda_M_arr * sigma_u**2
34
35 # --- Print numerical results ---
36 print(f"xi={xi}, theta={theta}, sigma_v={sigma_v}, sigma_u={sigma_u}")
37 print(f"{'I':>3} | {'chi_N':>10} | {'lambda_N':>10} | {'pi_N':>10} | {'chi_M'
38     ↪ ':>10} | {'lambda_M':>10} | {'pi_M':>10} | {'I_N':>12} | {'I_M':>12} | {'
39     ↪ W_N_N':>10} | {'W_N_M':>10}")
```

```

36 print("-" * 120)
37 for idx, I in enumerate(I_values):
38     print(f"{I:>3} | {chi_N_arr[idx]:>10.4f} | {lambda_N_arr[idx]:>10.6f} | "
39           f"{pi_N_arr[idx]:>10.4f} | {chi_M_arr[idx]:>10.4f} | "
40           f"{lambda_M_arr[idx]:>10.6f} | {pi_M_arr[idx]:>10.4f} | "
41           f"{I_info_N[idx]:>12.4f} | {I_info_M[idx]:>12.4f} | "
42           f"{W_N_nash[idx]:>10.6f} | {W_N_cartel[idx]:>10.6f}")
43
44 # --- Plots ---
45 fig, axes = plt.subplots(1, 3, figsize=(15, 5))
46
47 axes[0].plot(I_values, lambda_N_arr, 'b-o', label=r'$\lambda^N$')
48 axes[0].plot(I_values, lambda_M_arr, 'r--s', label=r'$\lambda^M$')
49 axes[0].set_xlabel(r'Number of speculators $I$')
50 axes[0].set_ylabel(r'Price impact $\lambda$')
51 axes[0].set_title(r'Price Impact $\lambda^N$ and $\lambda^M$ ($\xi=500$)')
52 axes[0].legend()
53 axes[0].grid(True, alpha=0.3)
54 axes[0].set_xticks(I_values)
55
56 axes[1].plot(I_values, I_info_N, 'b-o', label=r'$\mathcal{I}^N$')
57 axes[1].plot(I_values, I_info_M, 'r--s', label=r'$\mathcal{I}^M$')
58 axes[1].set_xlabel(r'Number of speculators $I$')
59 axes[1].set_ylabel(r'Price informativeness $\mathcal{I}$')
60 axes[1].set_title(r'Price Informativeness $\mathcal{I}^N$ and $\mathcal{I}^M$ (
    ↪ $\xi=500$)')
61 axes[1].legend()
62 axes[1].grid(True, alpha=0.3)
63 axes[1].set_xticks(I_values)
64
65 axes[2].plot(I_values, W_N_nash, 'b-o', label=r'$W_N^N$')
66 axes[2].plot(I_values, W_N_cartel, 'r--s', label=r'$W_N^M$')
67 axes[2].set_xlabel(r'Number of speculators $I$')
68 axes[2].set_ylabel(r'Noise-trader losses $W_N$')
69 axes[2].set_title(r'Noise-Trader Losses $W_N^N$ and $W_N^M$ ($\xi=500$)')
70 axes[2].legend()
71 axes[2].grid(True, alpha=0.3)
72 axes[2].set_xticks(I_values)
73
74 plt.tight_layout()
75 plt.savefig('comparative_statics_xi500.pdf', bbox_inches='tight')
76 plt.savefig('comparative_statics_xi500.png', bbox_inches='tight', dpi=150)
77 plt.close()
78 print("\nPlots saved.")

```

Output:

```

xi=500, theta=0.1, sigma_v=1.0, sigma_u=0.1
  I |      chi_N |    lambda_N |      pi_N |      chi_M |    lambda_M |
    ↪ pi_M |          I_N |          I_M |    W_N_N |    W_N_M
-----
    ↪

```

1	249.9999	0.002000	125.0000	249.9999	0.002000
	↪ 125.0000	6249996.0000	6249996.0000	0.000020	0.000020
2	166.6666	0.002000	55.5555	125.0000	0.002000
	↪ 62.5000	11111107.6667	6249996.0000	0.000020	0.000020
3	125.0000	0.002000	31.2500	83.3333	0.002000
	↪ 41.6667	14062497.2500	6249996.0000	0.000020	0.000020
4	100.0000	0.002000	20.0000	62.5000	0.002000
	↪ 31.2500	15999997.8000	6249996.0000	0.000020	0.000020
5	83.3333	0.002000	13.8889	50.0000	0.002000
	↪ 25.0000	17361109.3333	6249996.0000	0.000020	0.000020
6	71.4286	0.002000	10.2041	41.6667	0.002000
	↪ 20.8333	18367345.4898	6249996.0000	0.000020	0.000020
7	62.5000	0.002000	7.8125	35.7143	0.002000
	↪ 17.8571	19140623.8125	6249996.0000	0.000020	0.000020
8	55.5556	0.002000	6.1728	31.2500	0.002000
	↪ 15.6250	19753085.4444	6249996.0000	0.000020	0.000020
9	50.0000	0.002000	5.0000	27.7778	0.002000
	↪ 13.8889	20249999.2000	6249996.0000	0.000020	0.000020
10	45.4545	0.002000	4.1322	25.0000	0.002000
	↪ 12.5000	20661156.3719	6249996.0000	0.000020	0.000020

Plots saved.

A.5 Effective number of insiders I^*

```

1 import numpy as np
2 from scipy.optimize import brentq
3
4 # Parameters
5 xi_star = 500
6 theta_star = 0.1
7 sigma_v_star = 1.0
8 sigma_u_star = 0.1
9 I_cases = [2, 3, 5]
10
11 def effective_insiders_equation(I_star, lambda_M_target, sigma_v, sigma_u, xi,
12     ↪ theta):
13     """
14     Equation defining I*:  $\lambda^N(I^*) - \lambda^M(I) = 0$ .
15     Here I* is allowed to be non-integer.
16     """
17     _, lambda_N, _, _, _ = solve_equilibrium(I_star, sigma_v, sigma_u, xi,
18     ↪ theta)
19     return lambda_N - lambda_M_target
20
21 def count_sign_changes(values, tol=1e-10):
22     """
23     Count sign changes in a sequence, ignoring values very close to zero.
24     """
25     filtered_signs = []
26     for v in values:
27         if abs(v) < tol:

```

```

26         filtered_signs.append(0)
27     elif v > 0:
28         filtered_signs.append(1)
29     else:
30         filtered_signs.append(-1)
31
32     changes = 0
33     prev = None
34     for s in filtered_signs:
35         if s == 0:
36             continue
37         if prev is not None and s != prev:
38             changes += 1
39         prev = s
40     return changes
41
42 print("=" * 95)
43 print(f"Effective number of insiders I* for xi={xi_star}, theta={theta_star},
44       ↪ sigma_v={sigma_v_star}, sigma_u={sigma_u_star}")
45 print("=" * 95)
46 print(f"{'I':>5} | {'lambda^M(I)':>12} | {'I*':>12} | {'lambda^N(I*)':>14} | {'
47       ↪ Unique on grid?':>16}")
48 print("-" * 95)
49
50 for I in I_cases:
51     # Step 1: compute cartel price impact for this I
52     _, _, _, _, lambda_M, _ = solve_equilibrium(
53         I, sigma_v_star, sigma_u_star, xi_star, theta_star
54     )
55
56     # Step 2: define root function f(I*) = lambda^N(I*) - lambda^M(I)
57     f = lambda I_star: effective_insiders_equation(
58         I_star, lambda_M, sigma_v_star, sigma_u_star, xi_star, theta_star
59     )
60
61     # Step 3: bracket a root
62     a, b = 1e-6, 1.0
63     fa, fb = f(a), f(b)
64
65     while fa * fb > 0:
66         b *= 2.0
67         fb = f(b)
68         if b > 1e6:
69             raise ValueError(f"Could not find a bracket for I when I={I}")
70
71     # Step 4: solve for I*
72     I_star = brentq(f, a, b, xtol=1e-12)
73
74     # Step 5: verify numerically
75     _, lambda_N_check, _, _, _ = solve_equilibrium(
76         I_star, sigma_v_star, sigma_u_star, xi_star, theta_star
77     )

```

```

76
77     # Step 6: numerical uniqueness check on a grid
78     grid = np.linspace(0.01, 10.0, 2000)
79     f_vals = np.array([f(x) for x in grid])
80     sign_changes = count_sign_changes(f_vals)
81     near_zero_points = np.sum(np.abs(f_vals) < 1e-8)
82
83     # Heuristic uniqueness statement
84     unique_flag = "Yes" if sign_changes <= 1 else "Unclear"
85
86     print(f"{I:>5} | {lambda_M:>12.6f} | {I_star:>12.8f} | {lambda_N_check
      ↪ :>14.6f} | {unique_flag:>16}")

```

Output:

```

=====
      ↪
Effective number of insiders I* for xi=500, theta=0.1, sigma_v=1.0, sigma_u=0.1
=====
      ↪
      I |   lambda^M(I) |           I* |   lambda^N(I*) | Unique on grid?
      -----
      ↪
      2 |     0.002000 | 1.000000000 |     0.002000 |           Yes
      3 |     0.002000 | 1.000000000 |     0.002000 |           Yes
      5 |     0.002000 | 1.000000000 |     0.002000 |           Yes

```

A.6 Master function for Q-learning

```

1  import numpy as np
2  from scipy.stats import norm
3  from scipy.optimize import brentq
4  from numba import njit
5  import time
6  import matplotlib.pyplot as plt
7
8
9  @njit
10 def _nearest_idx(value, grid):
11     best = 0
12     best_diff = abs(grid[0] - value)
13     for k in range(1, grid.shape[0]):
14         d = abs(grid[k] - value)
15         if d < best_diff:
16             best_diff = d
17             best = k
18     return best
19
20
21 @njit
22 def _train_qlearning(Q, action_grids, price_grids, v_grid,
23                     T, alpha, beta, rho, lambda_N, sigma_u, v_bar,
24                     I, n_v, n_x, n_p, method_code, seed, report_every):

```



```

25     """method_code: 0=separate, 1=sequential, 2=averaged."""
26     np.random.seed(seed)
27
28     v_prev_idx = np.random.randint(0, n_v)
29     v_curr_idx = np.random.randint(0, n_v)
30     p_prev_idx = np.random.randint(0, n_p)
31
32     visit_count = np.zeros(n_v, dtype=np.int64)
33     n_reports = T // report_every
34     chi_history = np.empty(n_reports)
35     t_history = np.empty(n_reports, dtype=np.int64)
36     report_idx = 0
37
38     action_idx_arr = np.empty(I, dtype=np.int64)
39     x_arr = np.empty(I, dtype=np.float64)
40
41     # Per-cell stability tracking (needed for frac_stable_1e6)
42     I_eff = I if method_code == 0 else 1
43     stored_argmax = np.empty((I_eff, n_p, n_v, n_v), dtype=np.int64)
44     last_change_time = np.zeros((I_eff, n_p, n_v, n_v), dtype=np.int64)
45
46     for q_idx in range(I_eff):
47         for pp in range(n_p):
48             for vv in range(n_v):
49                 for kk in range(n_v):
50                     best_a = 0
51                     best_q = Q[q_idx, pp, vv, kk, 0]
52                     for a in range(1, n_x):
53                         if Q[q_idx, pp, vv, kk, a] > best_q:
54                             best_q = Q[q_idx, pp, vv, kk, a]
55                             best_a = a
56                     stored_argmax[q_idx, pp, vv, kk] = best_a
57
58     frac_stable_1e6 = np.empty(n_reports)
59
60     for t in range(T):
61         v_t = v_grid[v_curr_idx]
62         epsilon = np.exp(-beta * visit_count[v_curr_idx])
63
64         for i in range(I):
65             if np.random.random() < epsilon:
66                 action_idx_arr[i] = np.random.randint(0, n_x)
67             else:
68                 Q_idx = i if method_code == 0 else 0
69                 best_a = 0
70                 best_q = Q[Q_idx, p_prev_idx, v_prev_idx, v_curr_idx, 0]
71                 for a in range(1, n_x):
72                     q_val = Q[Q_idx, p_prev_idx, v_prev_idx, v_curr_idx, a]
73                     if q_val > best_q:
74                         best_q = q_val
75                         best_a = a
76                 action_idx_arr[i] = best_a

```

```

77     x_arr[i] = action_grids[v_curr_idx, action_idx_arr[i]]
78
79     u_t = np.random.normal(0.0, sigma_u)
80     y_t = 0.0
81     for i in range(I):
82         y_t += x_arr[i]
83     y_t += u_t
84     p_t = v_bar + lambda_N * y_t
85
86     v_next_idx = np.random.randint(0, n_v)
87     p_t_idx = _nearest_idx(p_t, price_grids[v_curr_idx])
88
89     # Q-update
90     if method_code == 0:
91         for i in range(I):
92             max_q_next = Q[i, p_t_idx, v_curr_idx, v_next_idx, 0]
93             for a in range(1, n_x):
94                 q_val = Q[i, p_t_idx, v_curr_idx, v_next_idx, a]
95                 if q_val > max_q_next:
96                     max_q_next = q_val
97             pi_it = (v_t - p_t) * x_arr[i]
98             target = pi_it + rho * max_q_next
99             old = Q[i, p_prev_idx, v_prev_idx, v_curr_idx, action_idx_arr[i]
100                 ↪ ]
101             Q[i, p_prev_idx, v_prev_idx, v_curr_idx, action_idx_arr[i]] = \
102                 (1.0 - alpha) * old + alpha * target
103
104     elif method_code == 1:
105         for i in range(I):
106             max_q_next = Q[0, p_t_idx, v_curr_idx, v_next_idx, 0]
107             for a in range(1, n_x):
108                 q_val = Q[0, p_t_idx, v_curr_idx, v_next_idx, a]
109                 if q_val > max_q_next:
110                     max_q_next = q_val
111             pi_it = (v_t - p_t) * x_arr[i]
112             target = pi_it + rho * max_q_next
113             old = Q[0, p_prev_idx, v_prev_idx, v_curr_idx, action_idx_arr[i]
114                 ↪ ]
115             Q[0, p_prev_idx, v_prev_idx, v_curr_idx, action_idx_arr[i]] = \
116                 (1.0 - alpha) * old + alpha * target
117
118     else: # averaged
119         max_q_next = Q[0, p_t_idx, v_curr_idx, v_next_idx, 0]
120         for a in range(1, n_x):
121             q_val = Q[0, p_t_idx, v_curr_idx, v_next_idx, a]
122             if q_val > max_q_next:
123                 max_q_next = q_val
124
125     if I == 2:
126         a0 = action_idx_arr[0]
127         a1 = action_idx_arr[1]
128         target_0 = (v_t - p_t) * x_arr[0] + rho * max_q_next

```

```

127     target_1 = (v_t - p_t) * x_arr[1] + rho * max_q_next
128     if a0 == a1:
129         avg_target = 0.5 * (target_0 + target_1)
130         old = Q[0, p_prev_idx, v_prev_idx, v_curr_idx, a0]
131         Q[0, p_prev_idx, v_prev_idx, v_curr_idx, a0] = \
132             (1.0 - alpha) * old + alpha * avg_target
133     else:
134         old0 = Q[0, p_prev_idx, v_prev_idx, v_curr_idx, a0]
135         Q[0, p_prev_idx, v_prev_idx, v_curr_idx, a0] = \
136             (1.0 - alpha) * old0 + alpha * target_0
137         old1 = Q[0, p_prev_idx, v_prev_idx, v_curr_idx, a1]
138         Q[0, p_prev_idx, v_prev_idx, v_curr_idx, a1] = \
139             (1.0 - alpha) * old1 + alpha * target_1
140     else:
141         for i in range(I):
142             pi_it = (v_t - p_t) * x_arr[i]
143             target = pi_it + rho * max_q_next
144             old = Q[0, p_prev_idx, v_prev_idx, v_curr_idx,
145                 ↪ action_idx_arr[i]]
146             Q[0, p_prev_idx, v_prev_idx, v_curr_idx, action_idx_arr[i]]
147                 ↪ = \
148                 (1.0 - alpha) * old + alpha * target
149
150     # Re-check argmax of just-updated cell, mark flips
151     for q_idx in range(I_eff):
152         best_a = 0
153         best_q = Q[q_idx, p_prev_idx, v_prev_idx, v_curr_idx, 0]
154         for a in range(1, n_x):
155             q_val = Q[q_idx, p_prev_idx, v_prev_idx, v_curr_idx, a]
156             if q_val > best_q:
157                 best_q = q_val
158                 best_a = a
159         if best_a != stored_argmax[q_idx, p_prev_idx, v_prev_idx,
160             ↪ v_curr_idx]:
161             stored_argmax[q_idx, p_prev_idx, v_prev_idx, v_curr_idx] =
162                 ↪ best_a
163             last_change_time[q_idx, p_prev_idx, v_prev_idx, v_curr_idx] = t
164                 ↪ + 1
165
166     visit_count[v_curr_idx] += 1
167     p_prev_idx = p_t_idx
168     v_prev_idx = v_curr_idx
169     v_curr_idx = v_next_idx
170
171     # Periodic diagnostic
172     if (t + 1) % report_every == 0 and report_idx < n_reports:
173         # 1) chi_history (stabilisation of bar chi^C)
174         chi_sum = 0.0
175         chi_count = 0
176         for k in range(n_v):
177             dv = v_grid[k] - v_bar
178             if abs(dv) > 1e-8:

```

```

174         if method_code == 0:
175             mean_a_over_i = 0.0
176             for i in range(I):
177                 a_sum = 0.0
178                 a_n = 0
179                 for pp in range(n_p):
180                     for vv in range(n_v):
181                         best_a = 0
182                         best_q = Q[i, pp, vv, k, 0]
183                         for a in range(1, n_x):
184                             q_val = Q[i, pp, vv, k, a]
185                             if q_val > best_q:
186                                 best_q = q_val
187                                 best_a = a
188                             a_sum += action_grids[k, best_a]
189                             a_n += 1
190                         mean_a_over_i += a_sum / a_n
191             mean_a = mean_a_over_i / I
192         else:
193             a_sum = 0.0
194             a_n = 0
195             for pp in range(n_p):
196                 for vv in range(n_v):
197                     best_a = 0
198                     best_q = Q[0, pp, vv, k, 0]
199                     for a in range(1, n_x):
200                         q_val = Q[0, pp, vv, k, a]
201                         if q_val > best_q:
202                             best_q = q_val
203                             best_a = a
204                         a_sum += action_grids[k, best_a]
205                         a_n += 1
206                     mean_a = a_sum / a_n
207             chi_sum += mean_a / dv
208             chi_count += 1
209         chi_history[report_idx] = chi_sum / chi_count
210         t_history[report_idx] = t + 1
211
212         # 2) Fraction of (s,i) cells stable for >= 10^6 consecutive periods
213         now = t + 1
214         n_total = I_eff * n_p * n_v * n_v
215         n_1e6 = 0
216         for q_idx in range(I_eff):
217             for pp in range(n_p):
218                 for vv in range(n_v):
219                     for kk in range(n_v):
220                         if now - last_change_time[q_idx, pp, vv, kk] >= 1
221                             ↪ _000_000:
222                             n_1e6 += 1
223         frac_stable_1e6[report_idx] = n_1e6 / n_total
224
225         report_idx += 1

```

```

225
226     return (chi_history[:report_idx],
227            t_history[:report_idx],
228            visit_count,
229            frac_stable_1e6[:report_idx])
230
231
232 @njit
233 def _evaluate_frozen_policy(Q, action_grids, price_grids, v_grid,
234                            T_eval, lambda_N, sigma_u, v_bar,
235                            I, n_v, n_x, n_p, method_code, seed):
236     np.random.seed(seed)
237     v_prev_idx = np.random.randint(0, n_v)
238     v_curr_idx = np.random.randint(0, n_v)
239     p_prev_idx = np.random.randint(0, n_p)
240     v_hist = np.empty(T_eval)
241     x_hist = np.empty((T_eval, I))
242     for t in range(T_eval):
243         v_t = v_grid[v_curr_idx]
244         v_hist[t] = v_t
245         x_total = 0.0
246         for i in range(I):
247             Q_idx = i if method_code == 0 else 0
248             best_a = 0
249             best_q = Q[Q_idx, p_prev_idx, v_prev_idx, v_curr_idx, 0]
250             for a in range(1, n_x):
251                 q_val = Q[Q_idx, p_prev_idx, v_prev_idx, v_curr_idx, a]
252                 if q_val > best_q:
253                     best_q = q_val
254                     best_a = a
255             x_it = action_grids[v_curr_idx, best_a]
256             x_hist[t, i] = x_it
257             x_total += x_it
258         u_t = np.random.normal(0.0, sigma_u)
259         p_t = v_bar + lambda_N * (x_total + u_t)
260         v_next_idx = np.random.randint(0, n_v)
261         p_t_idx = _nearest_idx(p_t, price_grids[v_curr_idx])
262         p_prev_idx = p_t_idx
263         v_prev_idx = v_curr_idx
264         v_curr_idx = v_next_idx
265     return v_hist, x_hist
266
267
268 # =====
269 # Equilibrium helpers
270 # =====
271
272 def _nash_residual(lam, I, sigma_v, sigma_u, theta, xi):
273     chi = 1.0 / ((I + 1) * lam)
274     gamma = (I * chi) / ((I * chi) ** 2 + (sigma_u / sigma_v) ** 2)
275     return lam - (theta * gamma + xi) / (theta + xi ** 2)
276

```

```

277 def _cartel_residual(lam, I, sigma_v, sigma_u, theta, xi):
278     chi = 1.0 / (2 * I * lam)
279     gamma = (I * chi) / ((I * chi) ** 2 + (sigma_u / sigma_v) ** 2)
280     return lam - (theta * gamma + xi) / (theta + xi ** 2)
281
282 def _solve_lambda(residual_fn, I, sigma_v, sigma_u, theta, xi):
283     lo, hi = 1e-10, 1.0
284     for _ in range(30):
285         try:
286             if residual_fn(lo, I, sigma_v, sigma_u, theta, xi) * \
287                 residual_fn(hi, I, sigma_v, sigma_u, theta, xi) < 0:
288                 break
289             except (ZeroDivisionError, OverflowError):
290                 pass
291             hi *= 10
292         else:
293             raise RuntimeError("Could not bracket lambda.")
294     return brentq(residual_fn, lo, hi, args=(I, sigma_v, sigma_u, theta, xi))
295
296 def _compute_equilibrium(I, sigma_v, sigma_u, theta, xi):
297     if xi == 0:
298         chi_N = sigma_u / (np.sqrt(I) * sigma_v)
299         chi_M = sigma_u / (I * sigma_v)
300         lambda_N = np.sqrt(I) * sigma_v / ((I + 1) * sigma_u)
301         lambda_M = sigma_v / (2 * sigma_u)
302     else:
303         lambda_N = _solve_lambda(_nash_residual, I, sigma_v, sigma_u, theta,
304             ↪ xi)
305         lambda_M = _solve_lambda(_cartel_residual, I, sigma_v, sigma_u, theta,
306             ↪ xi)
307         chi_N = 1.0 / ((I + 1) * lambda_N)
308         chi_M = 1.0 / (2 * I * lambda_M)
309     return chi_N, chi_M, lambda_N, lambda_M
310
311 # =====
312 # Master function
313 # =====
314
315 def run_qlearning(
316     *,
317     I=2, xi=0, method="separate",
318     T=int(50e6), T_eval=int(1e5),
319     v_bar=1.0, sigma_v=1.0, sigma_u=0.1,
320     rho=0.95, theta=0.1,
321     alpha=0.05, beta=5e-6,
322     iota=0.1, n_v=10, n_x=15, n_p=31,
323     seed_train=610014, seed_eval=987654,
324     save_plot=True, plot_dir=".", verbose=True,
325 ):
326     """
327     Convergence is reported via two metrics:

```

```

327     (1) chi_history : trace of bar chi^C over training (stabilisation)
328     (2) frac_stable_1e6 : fraction of (s,i) cells whose greedy action has
329         remained unchanged for >= 10^6 consecutive periods (softer
330         variant of the Dou et al. criterion)
331
332 Returns a dict including:
333     'chi_N', 'chi_M', 'lambda_N', 'lambda_M' - theoretical benchmarks
334     'bar_chi_ols', 'Delta_ols' - primary OLS estimator (IA
335         ↪ .4.4)
336     'bar_chi_ratio', 'Delta_ratio' - per-period ratio (diagnostic)
337     'chi_per_spec', 'intercept_per_spec' - per-speculator OLS detail
338     'Q', 'Q_list', 'Q_of' - Q-matrix + IRF-cell shim
339     'chi_history', 't_history' - bar chi^C trace
340     'frac_stable_1e6' - fraction stable >= 10^6 per
341         ↪ checkpoint
342     'v_hist', 'x_hist' - frozen-policy sample paths
343     'action_grids', 'price_grids', 'v_grid' - grids
344     'train_elapsed' - training wall time (s)
345
346 """
347 METHOD_CODE = {"separate": 0, "sequential": 1, "averaged": 2}[method]
348
349 if verbose:
350     print(f"==== Q-learning: I={I}, xi={xi}, method='{method}', T={T:},
351         ↪ =====")
352
353 chi_N, chi_M, lambda_N, lambda_M = _compute_equilibrium(
354     I, sigma_v, sigma_u, theta, xi)
355
356 if verbose:
357     print(f"\nBenchmarks (xi={xi}, I={I}):")
358     print(f"    chi^N    = {chi_N:.6f}")
359     print(f"    chi^M    = {chi_M:.6f}")
360     print(f"    lambda^N = {lambda_N:.6e}")
361     print(f"    lambda^M = {lambda_M:.6e}")
362
363 # Grids
364 k_arr = np.arange(1, n_v + 1)
365 v_grid = v_bar + sigma_v * norm.ppf((2 * k_arr - 1) / (2 * n_v))
366 x_N_k = chi_N * (v_grid - v_bar)
367 x_M_k = chi_M * (v_grid - v_bar)
368
369 action_grids = np.zeros((n_v, n_x))
370 for k in range(n_v):
371     xN_abs, xM_abs = abs(x_N_k[k]), abs(x_M_k[k])
372     width = xN_abs - xM_abs
373     pos_lower = xM_abs - iota * width
374     pos_upper = xN_abs + iota * width
375     if v_grid[k] > v_bar:
376         action_grids[k, :] = np.linspace(pos_lower, pos_upper, n_x)
377     else:
378         action_grids[k, :] = np.linspace(-pos_upper, -pos_lower, n_x)
379
380 price_grids = np.zeros((n_v, n_p))

```

```

376 for k in range(n_v):
377     p_H = v_bar + lambda_N * (I * max(x_M_k[k], x_N_k[k]) + 1.96 * sigma_u)
378     p_L = v_bar + lambda_N * (I * min(x_M_k[k], x_N_k[k]) - 1.96 * sigma_u)
379     price_grids[k, :] = np.linspace(p_L - iota * (p_H - p_L),
380                                     p_H + iota * (p_H - p_L), n_p)
381
382 # Q-init
383 Q_init = np.zeros((n_p, n_v, n_v, n_x))
384 for k in range(n_v):
385     v_k, X_k = v_grid[k], action_grids[k]
386     for n in range(n_x):
387         x_n = X_k[n]
388         total = 0.0
389         for m in range(n_x):
390             total += (v_k - (v_bar + lambda_N * (x_n + (I - 1) * X_k[m])))
391                     ↪ * x_n
392             Q_init[:, :, k, n] = total / ((1 - rho) * n_x)
393
394 Q = np.empty((I, n_p, n_v, n_v, n_x))
395 for i in range(I):
396     Q[i] = Q_init.copy()
397
398 # JIT warm-up
399 if verbose:
400     print("\nJIT warm-up (T=100)...")
401 t0 = time.time()
402 Q_warm = Q.copy()
403 _train_qlearning(Q_warm, action_grids, price_grids, v_grid,
404                 100, alpha, beta, rho, lambda_N, sigma_u, v_bar,
405                 I, n_v, n_x, n_p, METHOD_CODE, 0, 50)
406 _evaluate_frozen_policy(Q_warm, action_grids, price_grids, v_grid,
407                        100, lambda_N, sigma_u, v_bar,
408                        I, n_v, n_x, n_p, METHOD_CODE, 1)
409 if verbose:
410     print(f"Warm-up done in {time.time() - t0:.1f}s")
411
412 # Training
413 if verbose:
414     print(f"\nTRAINING for T = {T:,} periods...")
415 t0 = time.time()
416 report_every = max(T // 20, 1)
417 (chi_history, t_history, visit_count,
418  frac_1e6) = _train_qlearning(
419     Q, action_grids, price_grids, v_grid,
420     T, alpha, beta, rho, lambda_N, sigma_u, v_bar,
421     I, n_v, n_x, n_p, METHOD_CODE, seed_train, report_every
422 )
423 train_elapsed = time.time() - t0
424
425 if verbose:
426     print(f"Training time: {train_elapsed:.1f}s ({train_elapsed/60:.2f} min
427           ↪ )")

```



```

426     print(f"{'t':>14} | {'diag_chi':>9} | {'Delta':>7} | {'frac>=1e6':>10}"
         ↪ )
427     for j in range(len(t_history)):
428         dc = chi_history[j]
429         dD = ((chi_N - dc) / (chi_N - chi_M)
430               if abs(chi_N - chi_M) > 1e-12 else np.nan)
431         dD_str = "  n/a  " if np.isnan(dD) else f"{dD:7.4f}"
432         print(f"   t = {t_history[j]:>10,} | {dc:.6f} | {dD_str} | "
433               f"{100*frac_1e6[j]:9.2f}%")
434
435     # Frozen-policy evaluation
436     if verbose:
437         print(f"\nEVALUATION for T_eval = {T_eval:,} periods...")
438     t0 = time.time()
439     v_hist, x_hist = _evaluate_frozen_policy(
440         Q, action_grids, price_grids, v_grid,
441         T_eval, lambda_N, sigma_u, v_bar,
442         I, n_v, n_x, n_p, METHOD_CODE, seed_eval
443     )
444     if verbose:
445         print(f"Evaluation time: {time.time() - t0:.2f}s")
446
447     # Estimators
448     chi_per_spec = np.empty(I)
449     intercept_per_spec = np.empty(I)
450     for i in range(I):
451         slope, intercept = np.polyfit(v_hist, x_hist[:, i], 1)
452         chi_per_spec[i] = slope
453         intercept_per_spec[i] = intercept
454     bar_chi_ols = chi_per_spec.mean()
455
456     mask = np.abs(v_hist - v_bar) > 1e-12
457     x_bar_t = x_hist.mean(axis=1)
458     bar_chi_ratio = (x_bar_t[mask] / (v_hist[mask] - v_bar)).mean()
459
460     if abs(chi_N - chi_M) > 1e-12:
461         Delta_ols = (chi_N - bar_chi_ols) / (chi_N - chi_M)
462         Delta_ratio = (chi_N - bar_chi_ratio) / (chi_N - chi_M)
463     else:
464         Delta_ols = Delta_ratio = np.nan
465
466     # Report
467     if verbose:
468         print(f"\n==== Results (I={I}, xi={xi}, method='{method}') =====")
469         print(f"   chi^N = {chi_N:.6f}   chi^M = {chi_M:.6f}")
470         print(f"   --- Per-speculator OLS (primary, Dou et al. IA.4.4) ---")
471         for i in range(I):
472             implied = -v_bar * chi_per_spec[i]
473             mismatch = 100 * abs(intercept_per_spec[i] - implied) / max(abs(
474                 ↪ implied), 1e-12)
475             print(f"       Spec {i}: chi_hat = {chi_per_spec[i]:.6f} "
476                   f"(intercept mismatch = {mismatch:.3f}%)")

```

```

476     print(f"      bar_chi^C (OLS)      = {bar_chi_ols:.6f}")
477     if I >= 2 and abs(bar_chi_ols) > 1e-12:
478         asym = 100 * abs(chi_per_spec[0] - chi_per_spec[1]) / abs(
479             ↪ bar_chi_ols)
480         print(f"      Speculator asymmetry = {asym:.3f}% of mean")
481     if not np.isnan(Delta_ols):
482         print(f"      Delta_chi (OLS)      = {Delta_ols:.4f}      (0=Nash, 1=
483             ↪ cartel)")
484     else:
485         rel = abs(bar_chi_ols - chi_N) / chi_N * 100
486         print(f"      Rel. err vs chi^N      = {rel:.2f}%      (Delta undefined at
487             ↪ I=1)")
488     print(f"      --- Per-period ratio (diagnostic only) ---")
489     print(f"      bar_chi^C (ratio)      = {bar_chi_ratio:.6f}")
490     if not np.isnan(Delta_ratio):
491         print(f"      Delta_chi (ratio)      = {Delta_ratio:.4f}")
492
493     print(f"      --- Convergence ---")
494     print(f"      Cells stable >= 10^6 periods : {100*frac_1e6[-1]:.2f}%")
495
496     # Plot: per-cell stability over training
497     if save_plot:
498         try:
499             fig, ax = plt.subplots(figsize=(10, 6))
500             ax.plot(t_history, 100*frac_1e6, 'r-^', markersize=5,
501                 label=r'Cells stable for  $\geq 10^6$  consecutive periods')
502             ax.axhline(100, color='black', linestyle=':', alpha=0.5,
503                 label='Full convergence (Dou et al.)')
504             ax.set_xlabel('Period $t$')
505             ax.set_ylabel('Cells stable (% of total)')
506             ax.set_title(rf'Policy stabilisation ($I={I}$, $\xi={\xi}$, method
507                 ↪="{method}")')
508             ax.set_ylim(0, 105)
509             ax.legend(loc='lower right')
510             ax.grid(True, alpha=0.3)
511             plt.tight_layout()
512             fname = (f'{plot_dir}/convergence_I{I}_xi{xi}_{method}'
513                 f'_a{alpha}_b{beta:.0e}')
514             plt.savefig(f'{fname}.pdf', bbox_inches='tight')
515             plt.savefig(f'{fname}.png', bbox_inches='tight', dpi=150)
516             plt.close()
517             if verbose:
518                 print(f"\nPlot saved: {fname}.pdf / .png")
519         except Exception as e:
520             if verbose:
521                 print(f"\nPlotting skipped: {e}")
522
523     # Compatibility shim
524     if method == "separate":
525         Q_list = [Q[i] for i in range(I)]
526     else:
527         Q_list = [Q[0]]

```

```

524
525 def Q_of(agent_idx):
526     return Q_list[agent_idx] if method == "separate" else Q_list[0]
527
528     return {
529         'I': I,
530         'xi': xi,
531         'method': method,
532         'T': T,
533         'T_eval': T_eval,
534         'v_bar': v_bar,
535         'sigma_v': sigma_v,
536         'sigma_u': sigma_u,
537         'rho': rho,
538         'theta': theta,
539         'alpha': alpha,
540         'beta': beta,
541         'iota': iota,
542         'n_v': n_v,
543         'n_x': n_x,
544         'n_p': n_p,
545         'seed_train': seed_train,
546         'seed_eval': seed_eval,
547
548         'chi_N': chi_N, 'chi_M': chi_M,
549         'lambda_N': lambda_N, 'lambda_M': lambda_M,
550         'bar_chi_ols': bar_chi_ols, 'Delta_ols': Delta_ols,
551         'bar_chi_ratio': bar_chi_ratio, 'Delta_ratio': Delta_ratio,
552         'chi_per_spec': chi_per_spec, 'intercept_per_spec': intercept_per_spec,
553         'Q': Q, 'Q_list': Q_list, 'Q_of': Q_of,
554         'chi_history': chi_history, 't_history': t_history,
555         'frac_stable_1e6': frac_1e6,
556         'v_hist': v_hist, 'x_hist': x_hist,
557         'action_grids': action_grids, 'price_grids': price_grids, 'v_grid':
558             ↪ v_grid,
559         'train_elapsed': train_elapsed,
560     }

```

A.7 $I = 1$ sanity check

```

1 res_sanity = run_qlearning(I=1, xi=0, method="separate", T=int(10e6))

```

Output:

```

===== Q-learning: I=1, xi=0, method='separate', T=10,000,000 =====

```

```

Benchmarks (xi=0, I=1):
  chi^N      = 0.100000
  chi^M      = 0.100000
  lambda^N   = 5.000000e+00
  lambda^M   = 5.000000e+00

```

JIT warm-up (T=100)...

Warm-up done in 1.6s

TRAINING for T = 10,000,000 periods...

Training time: 1.5s (0.02 min)

	t		diag_chi		Delta		frac>=1e6
t =	500,000		0.100000		n/a		0.00%
t =	1,000,000		0.100000		n/a		0.13%
t =	1,500,000		0.100000		n/a		65.65%
t =	2,000,000		0.100000		n/a		69.48%
t =	2,500,000		0.100000		n/a		60.16%
t =	3,000,000		0.100000		n/a		50.10%
t =	3,500,000		0.100000		n/a		44.68%
t =	4,000,000		0.100000		n/a		40.45%
t =	4,500,000		0.100000		n/a		37.42%
t =	5,000,000		0.100000		n/a		35.13%
t =	5,500,000		0.100000		n/a		32.32%
t =	6,000,000		0.100000		n/a		30.45%
t =	6,500,000		0.100000		n/a		28.74%
t =	7,000,000		0.100000		n/a		27.16%
t =	7,500,000		0.100000		n/a		25.61%
t =	8,000,000		0.100000		n/a		24.87%
t =	8,500,000		0.100000		n/a		23.81%
t =	9,000,000		0.100000		n/a		23.19%
t =	9,500,000		0.100000		n/a		22.52%
t =	10,000,000		0.100000		n/a		21.87%

EVALUATION for T_eval = 100,000 periods...

Evaluation time: 0.01s

==== Results (I=1, xi=0, method='separate') =====

chi^N = 0.100000 chi^M = 0.100000

--- Per-speculator OLS (primary, Dou et al. IA.4.4) ---

Spec 0: chi_hat = 0.100000 (intercept mismatch = 0.000%)

bar_chi^C (OLS) = 0.100000

Rel. err vs chi^N = 0.00% (Delta undefined at I=1)

--- Per-period ratio (diagnostic only) ---

bar_chi^C (ratio) = 0.100000

--- Convergence ---

Cells stable >= 10^6 periods : 21.87%

Plot saved: ./convergence_I1_xi0_separate_a0.05_b5e-06.pdf / .png

A.8 $I = 2, \xi = 0$ method comparison

```
1 # --- xi=0, I=2 baseline + method comparison ---
2 results_by_method = {}
3 for m in ("separate", "sequential", "averaged"):
4     results_by_method[m] = run_qlearning(
5         I=2, xi=0, method=m, T=int(500e6), verbose=True
6     )
7 res_xi0 = results_by_method["separate"]
```

```

8
9 # --- Summary ---
10 print(f"\nNash:   chi^N = {res_xi0['chi_N']:.4f}")
11 print(f"Cartel: chi^M = {res_xi0['chi_M']:.4f}\n")
12 for m, res in results_by_method.items():
13     print(f"  {m:<12}  bar_chi^C = {res['bar_chi_ols']:.4f}  "
14           f"Delta = {res['Delta_ols']:.4f}")

```

Output:

```
===== Q-learning: I=2, xi=0, method='separate', T=500,000,000 =====
```

Benchmarks (xi=0, I=2):

```

chi^N      = 0.070711
chi^M      = 0.050000
lambda^N   = 4.714045e+00
lambda^M   = 5.000000e+00

```

JIT warm-up (T=100)...

Warm-up done in 0.0s

TRAINING for T = 500,000,000 periods...

Training time: 83.8s (1.40 min)

t	diag_chi	Delta	frac>=1e6
t = 25,000,000	0.060833	0.4769	15.73%
t = 50,000,000	0.060556	0.4903	19.19%
t = 75,000,000	0.059996	0.5174	53.31%
t = 100,000,000	0.059449	0.5438	72.29%
t = 125,000,000	0.058986	0.5661	88.68%
t = 150,000,000	0.058842	0.5731	92.65%
t = 175,000,000	0.058616	0.5840	90.61%
t = 200,000,000	0.058381	0.5954	97.47%
t = 225,000,000	0.058285	0.6000	97.10%
t = 250,000,000	0.058194	0.6043	97.47%
t = 275,000,000	0.057965	0.6154	97.74%
t = 300,000,000	0.058022	0.6127	98.03%
t = 325,000,000	0.057902	0.6185	98.65%
t = 350,000,000	0.057852	0.6209	98.94%
t = 375,000,000	0.057770	0.6248	98.95%
t = 400,000,000	0.057820	0.6224	98.53%
t = 425,000,000	0.057845	0.6212	99.10%
t = 450,000,000	0.057844	0.6213	98.52%
t = 475,000,000	0.057818	0.6225	97.76%
t = 500,000,000	0.057777	0.6245	98.52%

EVALUATION for T_eval = 100,000 periods...

Evaluation time: 0.01s

```
===== Results (I=2, xi=0, method='separate') =====
```

```
chi^N = 0.070711   chi^M = 0.050000
```

```
--- Per-speculator OLS (primary, Dou et al. IA.4.4) ---
```

```
Spec 0: chi_hat = 0.055840 (intercept mismatch = 0.137%)
```

```

Spec 1: chi_hat = 0.055778 (intercept mismatch = 0.003%)
bar_chi^C (OLS)      = 0.055809
Speculator asymmetry = 0.111% of mean
Delta_chi (OLS)      = 0.7195    (0=Nash, 1=cartel)
--- Per-period ratio (diagnostic only) ---
bar_chi^C (ratio)    = 0.057161
Delta_chi (ratio)    = 0.6543
--- Convergence ---
Cells stable >= 10^6 periods : 98.52%

Plot saved: ./convergence_I2_xi0_separate_a0.05_b5e-06.pdf / .png
===== Q-learning: I=2, xi=0, method='sequential', T=500,000,000 =====

Benchmarks (xi=0, I=2):
chi^N      = 0.070711
chi^M      = 0.050000
lambda^N   = 4.714045e+00
lambda^M   = 5.000000e+00

JIT warm-up (T=100)...
Warm-up done in 0.0s

TRAINING for T = 500,000,000 periods...
Training time: 70.6s (1.18 min)

```

t	diag_chi	Delta	frac>=1e6
t = 25,000,000	0.058493	0.5899	6.42%
t = 50,000,000	0.057218	0.6515	45.26%
t = 75,000,000	0.056277	0.6969	83.68%
t = 100,000,000	0.055916	0.7143	83.90%
t = 125,000,000	0.055676	0.7260	89.19%
t = 150,000,000	0.055400	0.7393	92.52%
t = 175,000,000	0.055441	0.7373	91.29%
t = 200,000,000	0.055137	0.7520	96.39%
t = 225,000,000	0.055003	0.7584	95.81%
t = 250,000,000	0.055141	0.7518	96.10%
t = 275,000,000	0.054928	0.7621	95.58%
t = 300,000,000	0.054860	0.7654	95.97%
t = 325,000,000	0.054771	0.7696	97.48%
t = 350,000,000	0.054708	0.7727	97.55%
t = 375,000,000	0.054706	0.7728	98.74%
t = 400,000,000	0.054660	0.7750	98.16%
t = 425,000,000	0.054699	0.7731	97.32%
t = 450,000,000	0.054582	0.7787	98.19%
t = 475,000,000	0.054599	0.7780	96.06%
t = 500,000,000	0.054520	0.7818	98.10%

```

EVALUATION for T_eval = 100,000 periods...
Evaluation time: 0.01s

===== Results (I=2, xi=0, method='sequential') =====
chi^N = 0.070711    chi^M = 0.050000
--- Per-speculator OLS (primary, Dou et al. IA.4.4) ---

```

```

Spec 0: chi_hat = 0.051687 (intercept mismatch = 0.177%)
Spec 1: chi_hat = 0.051687 (intercept mismatch = 0.177%)
bar_chi^C (OLS)      = 0.051687
Speculator asymmetry = 0.000% of mean
Delta_chi (OLS)      = 0.9186      (0=Nash, 1=cartel)
--- Per-period ratio (diagnostic only) ---
bar_chi^C (ratio)    = 0.054005
Delta_chi (ratio)    = 0.8066
--- Convergence ---
Cells stable >= 10^6 periods : 98.10%

Plot saved: ./convergence_I2_xi0_sequential_a0.05_b5e-06.pdf / .png
===== Q-learning: I=2, xi=0, method='averaged', T=500,000,000 =====

Benchmarks (xi=0, I=2):
chi^N      = 0.070711
chi^M      = 0.050000
lambda^N   = 4.714045e+00
lambda^M   = 5.000000e+00

JIT warm-up (T=100)...
Warm-up done in 0.0s

TRAINING for T = 500,000,000 periods...
Training time: 69.8s (1.16 min)

```

t	diag_chi	Delta	frac>=1e6
t = 25,000,000	0.059246	0.5536	8.00%
t = 50,000,000	0.058424	0.5933	11.42%
t = 75,000,000	0.057471	0.6393	46.42%
t = 100,000,000	0.056958	0.6640	67.16%
t = 125,000,000	0.056366	0.6926	82.48%
t = 150,000,000	0.056055	0.7076	89.03%
t = 175,000,000	0.055899	0.7152	86.48%
t = 200,000,000	0.055820	0.7190	92.87%
t = 225,000,000	0.055538	0.7326	95.23%
t = 250,000,000	0.055507	0.7341	95.10%
t = 275,000,000	0.055244	0.7468	95.29%
t = 300,000,000	0.055480	0.7354	95.61%
t = 325,000,000	0.055317	0.7433	97.74%
t = 350,000,000	0.055293	0.7444	97.42%
t = 375,000,000	0.055362	0.7411	97.35%
t = 400,000,000	0.055218	0.7481	97.23%
t = 425,000,000	0.055079	0.7548	98.23%
t = 450,000,000	0.054963	0.7604	97.87%
t = 475,000,000	0.055114	0.7531	96.55%
t = 500,000,000	0.054998	0.7587	98.35%

```

EVALUATION for T_eval = 100,000 periods...
Evaluation time: 0.01s

===== Results (I=2, xi=0, method='averaged') =====
chi^N = 0.070711   chi^M = 0.050000

```

```

--- Per-speculator OLS (primary, Dou et al. IA.4.4) ---
Spec 0: chi_hat = 0.052032 (intercept mismatch = 0.016%)
Spec 1: chi_hat = 0.052032 (intercept mismatch = 0.016%)
bar_chi^C (OLS)      = 0.052032
Speculator asymmetry = 0.000% of mean
Delta_chi (OLS)      = 0.9019    (0=Nash, 1=cartel)
--- Per-period ratio (diagnostic only) ---
bar_chi^C (ratio)    = 0.054560
Delta_chi (ratio)    = 0.7798
--- Convergence ---
Cells stable >= 10^6 periods : 98.35%

Plot saved: ./convergence_I2_xi0_averaged_a0.05_b5e-06.pdf / .png

Nash:   chi^N = 0.0707
Cartel: chi^M = 0.0500

separate      bar_chi^C = 0.0558   Delta = 0.7195
sequential    bar_chi^C = 0.0517   Delta = 0.9186
averaged      bar_chi^C = 0.0520   Delta = 0.9019

```

A.9 $I = 2, \xi = 500$ method comparison

```

1 # --- xi=500, I=2 baseline + method comparison ---
2 results_xi500 = {}
3 for m in ("separate", "sequential", "averaged"):
4     results_xi500[m] = run_qlearning(
5         I=2, xi=500, method=m, T=int(500e6), verbose=True
6     )
7 res_xi500 = results_xi500["separate"]
8
9 # --- Summary ---
10 print(f"\nNash:   chi^N = {res_xi500['chi_N']:.4f}")
11 print(f"Cartel:  chi^M = {res_xi500['chi_M']:.4f}\n")
12 for m, res in results_xi500.items():
13     print(f"    {m:<12} bar_chi^C = {res['bar_chi_ols']:.4f}   "
14           f"Delta = {res['Delta_ols']:.4f}")

```

Output:

```

===== Q-learning: I=2, xi=500, method='separate', T=500,000,000 =====

Benchmarks (xi=500, I=2):
  chi^N      = 166.666633
  chi^M      = 124.999950
  lambda^N   = 2.000000e-03
  lambda^M   = 2.000001e-03

JIT warm-up (T=100)...
Warm-up done in 0.0s

TRAINING for T = 500,000,000 periods...

```


Training time: 80.2s (1.34 min)

t	diag_chi	Delta	frac $\geq 1e6$
t = 25,000,000	146.632255	0.4808	8.11%
t = 50,000,000	147.718661	0.4548	9.63%
t = 75,000,000	147.923154	0.4498	30.18%
t = 100,000,000	148.891473	0.4266	42.60%
t = 125,000,000	149.464630	0.4128	61.61%
t = 150,000,000	150.575229	0.3862	71.23%
t = 175,000,000	150.469238	0.3887	80.31%
t = 200,000,000	150.529722	0.3873	86.39%
t = 225,000,000	150.790667	0.3810	85.53%
t = 250,000,000	151.318893	0.3683	94.21%
t = 275,000,000	151.444469	0.3653	93.19%
t = 300,000,000	150.973271	0.3766	95.26%
t = 325,000,000	150.687556	0.3835	97.16%
t = 350,000,000	150.941589	0.3774	95.05%
t = 375,000,000	151.093086	0.3738	97.65%
t = 400,000,000	150.910483	0.3781	96.71%
t = 425,000,000	150.966358	0.3768	97.08%
t = 450,000,000	151.427188	0.3657	96.02%
t = 475,000,000	151.208294	0.3710	91.34%
t = 500,000,000	151.408179	0.3662	96.50%

EVALUATION for $T_{eval} = 100,000$ periods...

Evaluation time: 0.01s

==== Results (I=2, xi=500, method='separate') =====

$\chi^N = 166.666633$ $\chi^M = 124.999950$

--- Per-speculator OLS (primary, Dou et al. IA.4.4) ---

Spec 0: $\chi_{hat} = 154.109664$ (intercept mismatch = 0.357%)

Spec 1: $\chi_{hat} = 153.497437$ (intercept mismatch = 0.343%)

$\bar{\chi}^C$ (OLS) = 153.803550

Speculator asymmetry = 0.398% of mean

Δ_{χ} (OLS) = 0.3087 (0=Nash, 1=cartel)

--- Per-period ratio (diagnostic only) ---

$\bar{\chi}^C$ (ratio) = 150.939312

Δ_{χ} (ratio) = 0.3775

--- Convergence ---

Cells stable $\geq 10^6$ periods : 96.50%

Plot saved: ./convergence_I2_xi500_separate_a0.05_b5e-06.pdf / .png

==== Q-learning: I=2, xi=500, method='sequential', T=500,000,000 =====

Benchmarks (xi=500, I=2):

$\chi^N = 166.666633$

$\chi^M = 124.999950$

$\lambda^N = 2.000000e-03$

$\lambda^M = 2.000001e-03$

JIT warm-up (T=100)...

Warm-up done in 0.0s

TRAINING for T = 500,000,000 periods...

Training time: 69.4s (1.16 min)

t	diag_chi	Delta	frac>=1e6
t = 25,000,000	144.213475	0.5389	31.48%
t = 50,000,000	142.465779	0.5808	57.81%
t = 75,000,000	141.340202	0.6078	89.03%
t = 100,000,000	141.973843	0.5926	74.74%
t = 125,000,000	141.507253	0.6038	93.03%
t = 150,000,000	141.754949	0.5979	92.26%
t = 175,000,000	141.396654	0.6065	95.39%
t = 200,000,000	141.037207	0.6151	97.23%
t = 225,000,000	141.261861	0.6097	95.19%
t = 250,000,000	141.181216	0.6116	96.58%
t = 275,000,000	141.018774	0.6155	96.81%
t = 300,000,000	141.227299	0.6105	95.77%
t = 325,000,000	141.295271	0.6089	97.29%
t = 350,000,000	140.973843	0.6166	97.39%
t = 375,000,000	141.072921	0.6142	97.45%
t = 400,000,000	141.026838	0.6154	96.00%
t = 425,000,000	140.506101	0.6279	97.97%
t = 450,000,000	140.303336	0.6327	98.77%
t = 475,000,000	140.280294	0.6333	98.26%
t = 500,000,000	140.612091	0.6253	98.13%

EVALUATION for T_eval = 100,000 periods...

Evaluation time: 0.01s

==== Results (I=2, xi=500, method='sequential') =====

chi^N = 166.666633 chi^M = 124.999950

--- Per-speculator OLS (primary, Dou et al. IA.4.4) ---

Spec 0: chi_hat = 130.600242 (intercept mismatch = 0.162%)

Spec 1: chi_hat = 130.600242 (intercept mismatch = 0.162%)

bar_chi^C (OLS) = 130.600242

Speculator asymmetry = 0.000% of mean

Delta_chi (OLS) = 0.8656 (0=Nash, 1=cartel)

--- Per-period ratio (diagnostic only) ---

bar_chi^C (ratio) = 137.226217

Delta_chi (ratio) = 0.7066

--- Convergence ---

Cells stable >= 10^6 periods : 98.13%

Plot saved: ./convergence_I2_xi500_sequential_a0.05_b5e-06.pdf / .png

==== Q-learning: I=2, xi=500, method='averaged', T=500,000,000 =====

Benchmarks (xi=500, I=2):

chi^N = 166.666633

chi^M = 124.999950

lambda^N = 2.000000e-03

lambda^M = 2.000001e-03

JIT warm-up (T=100)...

Warm-up done in 0.0s

TRAINING for T = 500,000,000 periods...

Training time: 65.5s (1.09 min)

t	diag_chi	Delta	frac>=1e6
t = 25,000,000	144.833291	0.5240	30.94%
t = 50,000,000	143.792968	0.5490	36.03%
t = 75,000,000	142.322922	0.5842	73.65%
t = 100,000,000	142.415088	0.5820	74.06%
t = 125,000,000	141.337898	0.6079	91.48%
t = 150,000,000	141.670848	0.5999	86.55%
t = 175,000,000	141.394350	0.6065	92.55%
t = 200,000,000	141.023382	0.6154	93.65%
t = 225,000,000	141.339050	0.6079	89.68%
t = 250,000,000	141.248036	0.6100	94.81%
t = 275,000,000	141.061400	0.6145	96.23%
t = 300,000,000	140.880525	0.6189	95.61%
t = 325,000,000	140.775686	0.6214	96.71%
t = 350,000,000	141.127069	0.6129	95.58%
t = 375,000,000	141.125917	0.6130	97.61%
t = 400,000,000	141.051032	0.6148	97.65%
t = 425,000,000	140.661631	0.6241	98.13%
t = 450,000,000	140.580985	0.6261	97.29%
t = 475,000,000	140.734211	0.6224	97.06%
t = 500,000,000	140.530294	0.6273	97.52%

EVALUATION for T_eval = 100,000 periods...

Evaluation time: 0.01s

==== Results (I=2, xi=500, method='averaged') =====

chi^N = 166.666633 chi^M = 124.999950

--- Per-speculator OLS (primary, Dou et al. IA.4.4) ---

Spec 0: chi_hat = 130.645157 (intercept mismatch = 0.210%)

Spec 1: chi_hat = 130.645157 (intercept mismatch = 0.210%)

bar_chi^C (OLS) = 130.645157

Speculator asymmetry = 0.000% of mean

Delta_chi (OLS) = 0.8645 (0=Nash, 1=cartel)

--- Per-period ratio (diagnostic only) ---

bar_chi^C (ratio) = 136.995538

Delta_chi (ratio) = 0.7121

--- Convergence ---

Cells stable >= 10^6 periods : 97.52%

Plot saved: ./convergence_I2_xi500_averaged_a0.05_b5e-06.pdf / .png

Nash: chi^N = 166.6666

Cartel: chi^M = 125.0000

separate bar_chi^C = 153.8036 Delta = 0.3087

sequential bar_chi^C = 130.6002 Delta = 0.8656

averaged bar_chi^C = 130.6452 Delta = 0.8645

A.10 Master function for IRF analysis

```
1 def run_irf_analysis(  
2     qres,  
3     *,  
4     Nsim=1000,  
5     H=9,  
6     shock_period=2,          # 0-indexed, so 2 means t=3  
7     shock_sizes_pct=(0.005, 0.025, 0.07),  
8     shock_labels=("Small (0.5%)", "Medium (2.5%)", "Large (7%)"),  
9     seed_cal=123,  
10    seed_irf=20260610,  
11    T_cal=20_000,  
12    save_plot=True,  
13    plot_dir=".",  
14    use_per_speculator_order_flow=True,  
15    verbose=True,  
16 ):  
17     """  
18     Noise-shock impulse response analysis.  
19  
20     Input  
21     ----  
22     qres : dict  
23           Output of run_qlearning(...)  
24  
25     Returns  
26     -----  
27     dict with:  
28         results  
29         E_abs_x  
30         shock_sizes  
31         t_axis  
32         filename_base  
33     """  
34  
35     import numpy as np  
36     import matplotlib.pyplot as plt  
37     import time  
38  
39     start = time.time()  
40  
41     Q_list = qres["Q_list"]  
42     action_grids = qres["action_grids"]  
43     price_grids = qres["price_grids"]  
44     v_grid = qres["v_grid"]  
45  
46     v_bar = qres.get("v_bar", 1.0) if "v_bar" in qres else 1.0  
47     sigma_u = qres.get("sigma_u", 0.1) if "sigma_u" in qres else 0.1  
48     lambda_N = qres["lambda_N"]  
49  
50     I = qres.get("I", None)  
51     method = qres.get("method", None)
```

```

52 xi = qres.get("xi", None)
53
54 # Fallbacks if not stored in qres
55 if I is None:
56     I = len(Q_list) if len(Q_list) > 1 else 2
57 if method is None:
58     method = "separate" if len(Q_list) == I else "shared"
59 if xi is None:
60     xi = "unknown"
61
62 n_v = len(v_grid)
63 n_p = price_grids.shape[1]
64
65 if verbose:
66     print(f"=== IRF analysis for xi={xi}, I={I}, method='{method}' ===\n")
67
68 def nearest_idx(value, grid):
69     return int(np.argmax(np.abs(grid - value)))
70
71 def Q_of(i):
72     return Q_list[i] if method == "separate" else Q_list[0]
73
74 # =====
75 # Step 1: compute E[|x_i|] under frozen greedy policy
76 # =====
77 if verbose:
78     print("Computing E[|x_i|] under frozen greedy policy...")
79
80 rng_cal = np.random.default_rng(seed_cal)
81
82 v_prev_idx = rng_cal.integers(0, n_v)
83 v_curr_idx = rng_cal.integers(0, n_v)
84 p_prev_idx = rng_cal.integers(0, n_p)
85
86 abs_x_samples = []
87
88 for _ in range(T_cal):
89     x_total = 0.0
90     x_arr = np.empty(I)
91
92     for i in range(I):
93         ai = np.argmax(Q_of(i)[p_prev_idx, v_prev_idx, v_curr_idx, :])
94         xi_i = action_grids[v_curr_idx, ai]
95         x_arr[i] = xi_i
96         x_total += xi_i
97
98     abs_x_samples.extend(np.abs(x_arr))
99
100 u_t = rng_cal.normal(0, sigma_u)
101 p_t = v_bar + lambda_N * (x_total + u_t)
102
103 v_next_idx = rng_cal.integers(0, n_v)

```

```

104     p_t_idx = nearest_idx(p_t, price_grids[v_curr_idx])
105
106     p_prev_idx = p_t_idx
107     v_prev_idx = v_curr_idx
108     v_curr_idx = v_next_idx
109
110     E_abs_x = float(np.mean(abs_x_samples))
111     shock_sizes = [pct * E_abs_x for pct in shock_sizes_pct]
112
113     if verbose:
114         print(f" E[|x_i|] = {E_abs_x:.6f}")
115         for pct, size in zip(shock_sizes_pct, shock_sizes):
116             print(f" Shock size {100*pct:.1f}% => u_shock = {size:.6f}")
117         print()
118
119     # =====
120     # Step 2: pre-draw common random numbers
121     # =====
122     rng_irf = np.random.default_rng(seed_irf)
123
124     v0_idx_arr = rng_irf.integers(0, n_v, size=Nsim)
125     p0_idx_arr = rng_irf.integers(0, n_p, size=Nsim)
126     v_seq_idx_arr = rng_irf.integers(0, n_v, size=(Nsim, H))
127     u_seq_arr = rng_irf.normal(0, sigma_u, size=(Nsim, H))
128     v_seq_arr = v_grid[v_seq_idx_arr]
129
130     # =====
131     # Step 3: frozen-policy simulator
132     # =====
133     def simulate_path(v_seq_idx, v_seq, u_seq, v0_idx, p0_idx, shock_size=0.0):
134         p_rec = np.empty(H)
135         x_rec = np.empty(H)
136         pi_avg_rec = np.empty(H)
137
138         p_prev_idx = p0_idx
139         v_prev_idx = v0_idx
140
141         for t in range(H):
142             v_curr_idx = v_seq_idx[t]
143             v_t = v_seq[t]
144
145             x_total = 0.0
146             x_arr = np.empty(I)
147
148             for i in range(I):
149                 ai = np.argmax(Q_of(i)[p_prev_idx, v_prev_idx, v_curr_idx, :])
150                 xi_i = action_grids[v_curr_idx, ai]
151                 x_arr[i] = xi_i
152                 x_total += xi_i
153
154             u_t = u_seq[t]
155             if t == shock_period and shock_size != 0.0:

```

```

156         u_t += shock_size * np.sign(v_t - v_bar)
157
158         p_t = v_bar + lambda_N * (x_total + u_t)
159
160         pi_avg = np.mean((v_t - p_t) * x_arr)
161
162         p_rec[t] = p_t
163
164         if use_per_speculator_order_flow:
165             x_rec[t] = np.mean(x_arr)
166         else:
167             x_rec[t] = x_total
168
169         pi_avg_rec[t] = pi_avg
170
171         p_t_idx = nearest_idx(p_t, price_grids[v_curr_idx])
172         p_prev_idx = p_t_idx
173         v_prev_idx = v_curr_idx
174
175         return p_rec, x_rec, pi_avg_rec
176
177     # =====
178     # Step 4: run baseline vs shocked simulations
179     # =====
180     results = {}
181
182     for s_label, s_size in zip(shock_labels, shock_sizes):
183         if verbose:
184             print(f"Running IRF for shock = {s_label} ...")
185
186         t0 = time.time()
187
188         p_base = np.empty((Nsim, H))
189         x_base = np.empty((Nsim, H))
190         pi_base = np.empty((Nsim, H))
191
192         p_shk = np.empty((Nsim, H))
193         x_shk = np.empty((Nsim, H))
194         pi_shk = np.empty((Nsim, H))
195
196         for n in range(Nsim):
197             pb, xb, pib = simulate_path(
198                 v_seq_idx_arr[n], v_seq_arr[n], u_seq_arr[n],
199                 v0_idx_arr[n], p0_idx_arr[n],
200                 shock_size=0.0,
201             )
202
203             ps, xs, pis = simulate_path(
204                 v_seq_idx_arr[n], v_seq_arr[n], u_seq_arr[n],
205                 v0_idx_arr[n], p0_idx_arr[n],
206                 shock_size=s_size,
207             )

```

```

208
209     p_base[n] = pb
210     x_base[n] = xb
211     pi_base[n] = pib
212
213     p_shk[n] = ps
214     x_shk[n] = xs
215     pi_shk[n] = pis
216
217     sgn = np.sign(v_seq_arr - v_bar)
218
219     p_tilde_b = (p_base - v_bar) * sgn
220     p_tilde_s = (p_shk - v_bar) * sgn
221
222     x_tilde_b = x_base * sgn
223     x_tilde_s = x_shk * sgn
224
225     p_b_mean = p_tilde_b.mean(axis=0)
226     p_s_mean = p_tilde_s.mean(axis=0)
227
228     x_b_mean = x_tilde_b.mean(axis=0)
229     x_s_mean = x_tilde_s.mean(axis=0)
230
231     pi_b_mean = pi_base.mean(axis=0)
232     pi_s_mean = pi_shk.mean(axis=0)
233
234     eps = 1e-12
235     p_pct = 100.0 * (p_s_mean - p_b_mean) / np.where(np.abs(p_b_mean) < eps
236     ↪ , np.nan, p_b_mean)
237     x_pct = 100.0 * (x_s_mean - x_b_mean) / np.where(np.abs(x_b_mean) < eps
238     ↪ , np.nan, x_b_mean)
239     pi_pct = 100.0 * (pi_s_mean - pi_b_mean) / np.where(np.abs(pi_b_mean) <
240     ↪ eps, np.nan, pi_b_mean)
241
242     results[s_label] = {
243         "p_pct": p_pct,
244         "x_pct": x_pct,
245         "pi_pct": pi_pct,
246         "p_b_mean": p_b_mean,
247         "x_b_mean": x_b_mean,
248         "pi_b_mean": pi_b_mean,
249         "p_s_mean": p_s_mean,
250         "x_s_mean": x_s_mean,
251         "pi_s_mean": pi_s_mean,
252     }
253
254     if verbose:
255         print(f" Done in {time.time() - t0:.1f}s")
256
257     # =====
258     # Step 5: plot
259     # =====

```



```

257 t_axis = np.arange(1, H + 1)
258 filename_base = f"{plot_dir}/irf_xi{xi}_{method}_I{I}"
259
260 if save_plot:
261     if verbose:
262         print("\nGenerating IRF plots...")
263
264     fig, axes = plt.subplots(1, 3, figsize=(18, 5))
265     markers = ["s", "o", "D"]
266
267     for (label, r), m in zip(results.items(), markers):
268         axes[0].plot(t_axis, r["p_pct"], marker=m, label=label, markersize
269             ↪ =6)
270     axes[0].axvline(shock_period + 1, color="gray", linestyle=":", alpha
271         ↪ =0.7, label="shock at $t=3$")
272     axes[0].axhline(0.0, color="black", linewidth=0.5)
273     axes[0].set_xlabel("$t$")
274     axes[0].set_ylabel("Deviation from baseline mean (%)")
275     axes[0].set_title(rf"Price IRF ($\xi={xi}$, method={method})")
276     axes[0].legend()
277     axes[0].grid(True, alpha=0.3)
278
279     for (label, r), m in zip(results.items(), markers):
280         axes[1].plot(t_axis, r["pi_pct"], marker=m, label=label, markersize
281             ↪ =6)
282     axes[1].axvline(shock_period + 1, color="gray", linestyle=":", alpha
283         ↪ =0.7)
284     axes[1].axhline(0.0, color="black", linewidth=0.5)
285     axes[1].set_xlabel("$t$")
286     axes[1].set_ylabel("Deviation from baseline mean (%)")
287     axes[1].set_title(rf"Per-speculator profit IRF ($\xi={xi}$, method={
288         ↪ method})")
289     axes[1].legend()
290     axes[1].grid(True, alpha=0.3)
291
292     x_title = "Per-speculator order flow" if use_per_speculator_order_flow
293         ↪ else "Total informed order flow"
294
295     for (label, r), m in zip(results.items(), markers):
296         axes[2].plot(t_axis, r["x_pct"], marker=m, label=label, markersize
297             ↪ =6)
298     axes[2].axvline(shock_period + 1, color="gray", linestyle=":", alpha
299         ↪ =0.7)
300     axes[2].axhline(0.0, color="black", linewidth=0.5)
301     axes[2].set_xlabel("$t$")
302     axes[2].set_ylabel("Deviation from baseline mean (%)")
303     axes[2].set_title(rf"{x_title} IRF ($\xi={xi}$, method={method})")
304     axes[2].legend()
305     axes[2].grid(True, alpha=0.3)
306
307     plt.tight_layout()
308     plt.savefig(f"{filename_base}.pdf", bbox_inches="tight")

```

```

301     plt.savefig(f"{filename_base}.png", bbox_inches="tight", dpi=150)
302     plt.close()
303
304     if verbose:
305         print(f"IRF plot saved as {filename_base}.pdf / .png")
306
307     # =====
308     # Step 6: numerical summary
309     # =====
310     if verbose:
311         print(f"\n==== IRF numerical summary for xi={xi}, method={method}, I={
            ↪ I} =====")
312
313         for label, r in results.items():
314             print(f"\nShock = {label}")
315             print("  t | price (%) | profit (%) | order flow (%)")
316             print("  ---|-----|-----|-----")
317             for t in range(H):
318                 print(
319                     f"    {t+1:>2} | {r['p_pct'][t]:>10.3f} | "
320                     f"{r['pi_pct'][t]:>10.3f} | {r['x_pct'][t]:>10.3f}"
321                 )
322
323             print(f"\nTotal IRF runtime: {time.time() - start:.1f}s")
324
325     return {
326         "results": results,
327         "E_abs_x": E_abs_x,
328         "shock_sizes": shock_sizes,
329         "shock_sizes_pct": shock_sizes_pct,
330         "shock_labels": shock_labels,
331         "t_axis": t_axis,
332         "filename_base": filename_base,
333         "use_per_speculator_order_flow": use_per_speculator_order_flow,
334     }

```

A.11 IRF analysis at $\xi = 0$

```

1  # --- IRF analysis: xi=0, I=2, all methods ---
2  irf_xi0 = {}
3
4  for m, res in results_by_method.items():
5      print(f"\n==== IRF: xi=0, method='{m}' =====")
6
7      irf_xi0[m] = run_irf_analysis(
8          res,
9          verbose=True,
10         save_plot=True,
11         plot_dir="."
12     )

```

Output:

```
===== IRF: xi=0, method='separate' =====
=== IRF analysis for xi=0, I=2, method='separate' ===

Computing E[|x_i|] under frozen greedy policy...
E[|x_i|] = 0.043639
Shock size 0.5% => u_shock = 0.000218
Shock size 2.5% => u_shock = 0.001091
Shock size 7.0% => u_shock = 0.003055

Running IRF for shock = Small (0.5%) ...
Done in 0.2s
Running IRF for shock = Medium (2.5%) ...
Done in 0.2s
Running IRF for shock = Large (7%) ...
Done in 0.2s

Generating IRF plots...
IRF plot saved as ./irf_xi0_separate_I2.pdf / .png

===== IRF numerical summary for xi=0, method=separate, I=2 =====

Shock = Small (0.5%)
  t | price (%) | profit (%) | order flow (%)
  ---|-----|-----|-----
  1 | 0.000 | 0.000 | 0.000
  2 | 0.000 | 0.000 | 0.000
  3 | 0.261 | -0.180 | 0.000
  4 | 0.015 | 0.001 | 0.015
  5 | 0.000 | 0.000 | 0.000
  6 | 0.000 | 0.000 | 0.000
  7 | 0.000 | 0.000 | 0.000
  8 | 0.000 | 0.000 | 0.000
  9 | 0.000 | 0.000 | 0.000

Shock = Medium (2.5%)
  t | price (%) | profit (%) | order flow (%)
  ---|-----|-----|-----
  1 | 0.000 | 0.000 | 0.000
  2 | 0.000 | 0.000 | 0.000
  3 | 1.307 | -0.898 | 0.000
  4 | -0.027 | -0.152 | -0.027
  5 | -0.058 | -0.026 | -0.062
  6 | 0.070 | -0.005 | 0.069
  7 | -0.017 | -0.000 | -0.017
  8 | -0.008 | 0.007 | -0.008
  9 | 0.000 | 0.000 | 0.000

Shock = Large (7%)
  t | price (%) | profit (%) | order flow (%)
  ---|-----|-----|-----
```

1		0.000		0.000		0.000
2		0.000		0.000		0.000
3		3.660		-2.516		0.000
4		-0.212		-0.088		-0.212
5		-0.073		-0.099		-0.077
6		0.093		-0.129		0.091
7		0.007		-0.012		0.007
8		-0.026		0.010		-0.027
9		-0.001		-0.001		-0.001

Total IRF runtime: 1.4s

==== IRF: xi=0, method='sequential' =====
 == IRF analysis for xi=0, I=2, method='sequential' ==

Computing $E[|x_i|]$ under frozen greedy policy...

$E[|x_i|] = 0.040686$
 Shock size 0.5% => $u_{shock} = 0.000203$
 Shock size 2.5% => $u_{shock} = 0.001017$
 Shock size 7.0% => $u_{shock} = 0.002848$

Running IRF for shock = Small (0.5%) ...
 Done in 0.2s
 Running IRF for shock = Medium (2.5%) ...
 Done in 0.2s
 Running IRF for shock = Large (7%) ...
 Done in 0.2s

Generating IRF plots...
 IRF plot saved as ./irf_xi0_sequential_I2.pdf / .png

==== IRF numerical summary for xi=0, method=sequential, I=2 =====

Shock = Small (0.5%)

t		price (%)		profit (%)		order flow (%)
---		-----		-----		-----
1		0.000		0.000		0.000
2		0.000		0.000		0.000
3		0.265		-0.154		0.000
4		-0.022		-0.005		-0.022
5		0.043		0.002		0.045
6		0.002		-0.001		0.002
7		0.000		0.000		0.000
8		0.000		0.000		0.000
9		0.000		0.000		0.000

Shock = Medium (2.5%)

t		price (%)		profit (%)		order flow (%)
---		-----		-----		-----
1		0.000		0.000		0.000
2		0.000		0.000		0.000
3		1.325		-0.770		0.000

4		-0.060		-0.061		-0.060
5		0.105		-0.052		0.111
6		-0.010		-0.000		-0.010
7		0.000		0.000		0.000
8		0.000		0.000		0.000
9		0.000		0.000		0.000

Shock = Large (7%)

t		price (%)		profit (%)		order flow (%)
---		-----		-----		-----
1		0.000		0.000		0.000
2		0.000		0.000		0.000
3		3.711		-2.156		0.000
4		0.204		-0.125		0.204
5		0.335		-0.122		0.355
6		0.018		0.020		0.017
7		-0.001		-0.005		-0.001
8		0.000		0.000		0.000
9		0.000		0.000		0.000

Total IRF runtime: 1.5s

==== IRF: xi=0, method='averaged' =====
 == IRF analysis for xi=0, I=2, method='averaged' ==

Computing $E[|x_i|]$ under frozen greedy policy...

$E[|x_i|] = 0.041000$
 Shock size 0.5% => $u_{\text{shock}} = 0.000205$
 Shock size 2.5% => $u_{\text{shock}} = 0.001025$
 Shock size 7.0% => $u_{\text{shock}} = 0.002870$

Running IRF for shock = Small (0.5%) ...

Done in 0.2s

Running IRF for shock = Medium (2.5%) ...

Done in 0.2s

Running IRF for shock = Large (7%) ...

Done in 0.2s

Generating IRF plots...

IRF plot saved as ./irf_xi0_averaged_I2.pdf / .png

==== IRF numerical summary for xi=0, method=averaged, I=2 =====

Shock = Small (0.5%)

t		price (%)		profit (%)		order flow (%)
---		-----		-----		-----
1		0.000		0.000		0.000
2		0.000		0.000		0.000
3		0.262		-0.158		0.000
4		0.014		0.019		0.014
5		0.019		-0.030		0.020
6		-0.009		0.004		-0.009

7		0.000		0.000		0.000
8		0.000		0.000		0.000
9		0.000		0.000		0.000

Shock = Medium (2.5%)

t		price (%)		profit (%)		order flow (%)
---		-----		-----		-----
1		0.000		0.000		0.000
2		0.000		0.000		0.000
3		1.310		-0.792		0.000
4		-0.031		-0.211		-0.031
5		0.049		-0.058		0.052
6		0.008		-0.012		0.007
7		-0.036		0.009		-0.035
8		-0.009		-0.005		-0.009
9		0.006		-0.001		0.006

Shock = Large (7%)

t		price (%)		profit (%)		order flow (%)
---		-----		-----		-----
1		0.000		0.000		0.000
2		0.000		0.000		0.000
3		3.668		-2.218		0.000
4		-0.327		-0.240		-0.327
5		0.068		0.053		0.073
6		-0.047		-0.074		-0.045
7		-0.018		-0.037		-0.018
8		-0.030		0.007		-0.031
9		0.008		0.000		0.007

Total IRF runtime: 1.4s

A.12 IRF analysis at $\xi = 500$

```

1 # --- IRF analysis: xi=500, I=2, all methods ---
2 irf_xi500 = {}
3
4 for m, res in results_xi500.items():
5     print(f"\n==== IRF: xi=500, method='{m}' =====")
6
7     irf_xi500[m] = run_irf_analysis(
8         res,
9         verbose=True,
10        save_plot=True,
11        plot_dir="."
12    )

```

Output:

```

==== IRF: xi=500, method='separate' =====
=== IRF analysis for xi=500, I=2, method='separate' ===

```

```

Computing E[|x_i|] under frozen greedy policy...
E[|x_i|] = 119.020678
Shock size 0.5% => u_shock = 0.595103
Shock size 2.5% => u_shock = 2.975517
Shock size 7.0% => u_shock = 8.331447

Running IRF for shock = Small (0.5%) ...
Done in 0.2s
Running IRF for shock = Medium (2.5%) ...
Done in 0.2s
Running IRF for shock = Large (7%) ...
Done in 0.2s

Generating IRF plots...
IRF plot saved as ./irf_xi500_separate_I2.pdf / .png

===== IRF numerical summary for xi=500, method=separate, I=2 =====

Shock = Small (0.5%)
  t | price (%) | profit (%) | order flow (%)
  ---|-----|-----|-----
  1 | 0.000 | 0.000 | 0.000
  2 | 0.000 | 0.000 | 0.000
  3 | 0.246 | -0.266 | 0.000
  4 | -0.048 | 0.004 | -0.048
  5 | 0.484 | -0.290 | 0.484
  6 | -0.076 | -0.008 | -0.076
  7 | -0.028 | 0.013 | -0.028
  8 | -0.058 | -0.007 | -0.058
  9 | -0.057 | 0.037 | -0.057

Shock = Medium (2.5%)
  t | price (%) | profit (%) | order flow (%)
  ---|-----|-----|-----
  1 | 0.000 | 0.000 | 0.000
  2 | 0.000 | 0.000 | 0.000
  3 | 1.228 | -1.331 | 0.000
  4 | 0.212 | -0.080 | 0.212
  5 | 0.606 | -0.455 | 0.606
  6 | 0.468 | -0.415 | 0.468
  7 | 0.115 | -0.049 | 0.115
  8 | 0.063 | -0.078 | 0.063
  9 | 0.411 | -0.270 | 0.411

Shock = Large (7%)
  t | price (%) | profit (%) | order flow (%)
  ---|-----|-----|-----
  1 | 0.000 | 0.000 | 0.000
  2 | 0.000 | 0.000 | 0.000
  3 | 3.439 | -3.726 | 0.000
  4 | 0.132 | -0.039 | 0.132

```

5	0.589		-0.460		0.589
6	0.258		-0.139		0.258
7	-0.032		0.001		-0.032
8	-0.289		0.064		-0.289
9	0.222		-0.126		0.222

Total IRF runtime: 1.5s

==== IRF: xi=500, method='sequential' =====
 == IRF analysis for xi=500, I=2, method='sequential' ==

Computing $E[|x_i|]$ under frozen greedy policy...

$E[|x_i|] = 103.190489$
 Shock size 0.5% => $u_{\text{shock}} = 0.515952$
 Shock size 2.5% => $u_{\text{shock}} = 2.579762$
 Shock size 7.0% => $u_{\text{shock}} = 7.223334$

Running IRF for shock = Small (0.5%) ...

Done in 0.2s

Running IRF for shock = Medium (2.5%) ...

Done in 0.2s

Running IRF for shock = Large (7%) ...

Done in 0.2s

Generating IRF plots...

IRF plot saved as ./irf_xi500_sequential_I2.pdf / .png

==== IRF numerical summary for xi=500, method=sequential, I=2 =====

Shock = Small (0.5%)

t	price (%)	profit (%)	order flow (%)
---	-----	-----	-----
1	0.000		0.000
2	0.000		0.000
3	0.246		0.000
4	1.079		1.079
5	-0.150		-0.150
6	0.148		0.148
7	0.070		0.070
8	-0.291		-0.291
9	-0.086		-0.086

Shock = Medium (2.5%)

t	price (%)	profit (%)	order flow (%)
---	-----	-----	-----
1	0.000		0.000
2	0.000		0.000
3	1.229		0.000
4	4.402		4.402
5	-0.023		-0.023
6	-0.247		-0.247
7	-0.100		-0.100


```

      8 |      -0.001 |      0.009 |      -0.001
      9 |      -0.007 |      0.013 |      -0.007

Shock = Large (7%)
  t | price (%) | profit (%) | order flow (%)
  ---|-----|-----|-----
  1 |      0.000 |      0.000 |      0.000
  2 |      0.000 |      0.000 |      0.000
  3 |      3.440 |     -2.646 |      0.000
  4 |      4.244 |     -1.559 |      4.244
  5 |      0.090 |     -0.028 |      0.090
  6 |     -0.304 |      0.104 |     -0.304
  7 |     -0.238 |      0.041 |     -0.238
  8 |     -0.063 |      0.021 |     -0.063
  9 |     -0.048 |      0.033 |     -0.048

Total IRF runtime: 1.3s

===== IRF: xi=500, method='averaged' =====
=== IRF analysis for xi=500, I=2, method='averaged' ===

Computing E[|x_i|] under frozen greedy policy...
E[|x_i|] = 103.200123
Shock size 0.5% => u_shock = 0.516001
Shock size 2.5% => u_shock = 2.580003
Shock size 7.0% => u_shock = 7.224009

Running IRF for shock = Small (0.5%) ...
Done in 0.2s
Running IRF for shock = Medium (2.5%) ...
Done in 0.2s
Running IRF for shock = Large (7%) ...
Done in 0.2s

Generating IRF plots...
IRF plot saved as ./irf_xi500_averaged_I2.pdf / .png

===== IRF numerical summary for xi=500, method=averaged, I=2 =====

Shock = Small (0.5%)
  t | price (%) | profit (%) | order flow (%)
  ---|-----|-----|-----
  1 |      0.000 |      0.000 |      0.000
  2 |      0.000 |      0.000 |      0.000
  3 |      0.246 |     -0.188 |      0.000
  4 |      1.704 |     -0.515 |      1.704
  5 |     -0.130 |      0.011 |     -0.130
  6 |     -0.225 |      0.025 |     -0.225
  7 |     -0.037 |      0.004 |     -0.037
  8 |     -0.033 |      0.004 |     -0.033
  9 |     -0.390 |      0.065 |     -0.390

```

Shock = Medium (2.5%)

t	price (%)	profit (%)	order flow (%)
1	0.000	0.000	0.000
2	0.000	0.000	0.000
3	1.232	-0.942	0.000
4	5.074	-1.868	5.074
5	-0.190	0.019	-0.190
6	0.078	-0.022	0.078
7	0.162	-0.042	0.162
8	0.369	-0.071	0.369
9	-0.419	0.083	-0.419

Shock = Large (7%)

t	price (%)	profit (%)	order flow (%)
1	0.000	0.000	0.000
2	0.000	0.000	0.000
3	3.450	-2.638	0.000
4	4.387	-1.635	4.387
5	0.169	-0.029	0.169
6	0.047	0.008	0.047
7	0.285	-0.094	0.285
8	0.244	-0.037	0.244
9	-0.327	0.087	-0.327

Total IRF runtime: 1.2s

A.13 Multi-seed verification (separate matrices, $\xi = 500$)

```

1 # Multi-seed check for separate-matrices at xi=500
2 results_xi500_seeds = {}
3 seeds = [610014, 42, 123, 2026, 7777] # the first one matches your original
    ↪ run
4
5 for s in seeds:
6     print(f"\n==== seed_train={s} =====")
7     results_xi500_seeds[s] = run_qlearning(
8         I=2,
9         xi=500,
10        method='separate',
11        T=int(500e6),
12        seed_train=s,
13        seed_eval=s + 1, # use a different but linked seed for evaluation
14        save_plot=False, # skip plot to save disk
15        verbose=True,
16    )
17
18 # Summary
19 print("\n==== Summary =====")
20 print(f"{'seed':>10} | {'bar_chi^C':>10} | {'Delta_chi':>10} | {'frac_stable
    ↪ ':>12}")

```

```

21 print("-" * 55)
22 for s, res in results_xi500_seeds.items():
23     print(f"{s:>10} | {res['bar_chi_ols']:>10.4f} | "
24           f"{res['Delta_ols']:>10.4f} | {res['frac_stable_1e6'][-1]*100:>11.2f}
           ↪ }%")
25
26 # Compute mean and std of Delta_chi
27 deltas = np.array([r['Delta_ols'] for r in results_xi500_seeds.values()])
28 print(f"\nDelta_chi: mean = {deltas.mean():.4f}, std = {deltas.std():.4f}")
29 print(f"95% CI (assuming normality): [{deltas.mean() - 1.96*deltas.std()/np.
           ↪ sqrt(len(deltas)):.4f}, "
30       f"{deltas.mean() + 1.96*deltas.std()/np.sqrt(len(deltas)):.4f}]" )

```

Output:

```

===== seed_train=610014 =====
===== Q-learning: I=2, xi=500, method='separate', T=500,000,000 =====

```

Benchmarks (xi=500, I=2):

```

chi^N    = 166.666633
chi^M    = 124.999950
lambda^N = 2.000000e-03
lambda^M = 2.000001e-03

```

JIT warm-up (T=100)...

Warm-up done in 0.0s

TRAINING for T = 500,000,000 periods...

Training time: 76.3s (1.27 min)

t	diag_chi	Delta	frac>=1e6
t = 25,000,000	146.632255	0.4808	8.11%
t = 50,000,000	147.718661	0.4548	9.63%
t = 75,000,000	147.923154	0.4498	30.18%
t = 100,000,000	148.891473	0.4266	42.60%
t = 125,000,000	149.464630	0.4128	61.61%
t = 150,000,000	150.575229	0.3862	71.23%
t = 175,000,000	150.469238	0.3887	80.31%
t = 200,000,000	150.529722	0.3873	86.39%
t = 225,000,000	150.790667	0.3810	85.53%
t = 250,000,000	151.318893	0.3683	94.21%
t = 275,000,000	151.444469	0.3653	93.19%
t = 300,000,000	150.973271	0.3766	95.26%
t = 325,000,000	150.687556	0.3835	97.16%
t = 350,000,000	150.941589	0.3774	95.05%
t = 375,000,000	151.093086	0.3738	97.65%
t = 400,000,000	150.910483	0.3781	96.71%
t = 425,000,000	150.966358	0.3768	97.08%
t = 450,000,000	151.427188	0.3657	96.02%
t = 475,000,000	151.208294	0.3710	91.34%
t = 500,000,000	151.408179	0.3662	96.50%

```

EVALUATION for T_eval = 100,000 periods...
Evaluation time: 0.01s

===== Results (I=2, xi=500, method='separate') =====
chi^N = 166.666633   chi^M = 124.999950
--- Per-speculator OLS (primary, Dou et al. IA.4.4) ---
Spec 0: chi_hat = 154.169161 (intercept mismatch = 0.388%)
Spec 1: chi_hat = 153.577080 (intercept mismatch = 0.377%)
bar_chi^C (OLS)      = 153.873120
Speculator asymmetry = 0.385% of mean
Delta_chi (OLS)      = 0.3070   (0=Nash, 1=cartel)
--- Per-period ratio (diagnostic only) ---
bar_chi^C (ratio)    = 150.961919
Delta_chi (ratio)    = 0.3769
--- Convergence ---
Cells stable >= 10^6 periods : 96.50%

===== seed_train=42 =====
===== Q-learning: I=2, xi=500, method='separate', T=500,000,000 =====

Benchmarks (xi=500, I=2):
chi^N      = 166.666633
chi^M      = 124.999950
lambda^N   = 2.000000e-03
lambda^M   = 2.000001e-03

JIT warm-up (T=100)...
Warm-up done in 0.0s

TRAINING for T = 500,000,000 periods...
Training time: 76.9s (1.28 min)

```

t	diag_chi	Delta	frac>=1e6
t = 25,000,000	147.199652	0.4672	7.90%
t = 50,000,000	147.561403	0.4585	9.35%
t = 75,000,000	148.395505	0.4385	29.97%
t = 100,000,000	148.463477	0.4369	51.23%
t = 125,000,000	149.726150	0.4066	60.95%
t = 150,000,000	150.840206	0.3798	65.90%
t = 175,000,000	150.555644	0.3867	76.24%
t = 200,000,000	150.067164	0.3984	83.73%
t = 225,000,000	150.447925	0.3892	89.24%
t = 250,000,000	150.672003	0.3839	93.63%
t = 275,000,000	150.468662	0.3888	96.08%
t = 300,000,000	150.522234	0.3875	95.19%
t = 325,000,000	150.310828	0.3925	96.69%
t = 350,000,000	150.451957	0.3892	97.16%
t = 375,000,000	150.526266	0.3874	96.87%
t = 400,000,000	150.686980	0.3835	95.21%
t = 425,000,000	151.026266	0.3754	96.08%
t = 450,000,000	150.985943	0.3763	92.42%
t = 475,000,000	150.541243	0.3870	85.34%
t = 500,000,000	151.057948	0.3746	92.63%

```

EVALUATION for T_eval = 100,000 periods...
Evaluation time: 0.01s

==== Results (I=2, xi=500, method='separate') ====
chi^N = 166.666633   chi^M = 124.999950
--- Per-speculator OLS (primary, Dou et al. IA.4.4) ---
Spec 0: chi_hat = 155.027728 (intercept mismatch = 0.446%)
Spec 1: chi_hat = 152.834887 (intercept mismatch = 0.293%)
bar_chi^C (OLS)      = 153.931308
Speculator asymmetry = 1.425% of mean
Delta_chi (OLS)      = 0.3056   (0=Nash, 1=cartel)
--- Per-period ratio (diagnostic only) ---
bar_chi^C (ratio)    = 151.124026
Delta_chi (ratio)    = 0.3730
--- Convergence ---
Cells stable >= 10^6 periods : 92.63%

==== seed_train=123 ====
==== Q-learning: I=2, xi=500, method='separate', T=500,000,000 ====

Benchmarks (xi=500, I=2):
chi^N      = 166.666633
chi^M      = 124.999950
lambda^N   = 2.000000e-03
lambda^M   = 2.000001e-03

JIT warm-up (T=100)...
Warm-up done in 0.0s

TRAINING for T = 500,000,000 periods...
Training time: 76.9s (1.28 min)

```

t	diag_chi	Delta	frac>=1e6
t = 25,000,000	147.184675	0.4676	8.39%
t = 50,000,000	147.863246	0.4513	9.95%
t = 75,000,000	148.771657	0.4295	32.08%
t = 100,000,000	149.432371	0.4136	60.63%
t = 125,000,000	150.052763	0.3987	58.13%
t = 150,000,000	150.083869	0.3980	70.05%
t = 175,000,000	150.836750	0.3799	70.05%
t = 200,000,000	150.515321	0.3876	75.81%
t = 225,000,000	150.756681	0.3818	87.85%
t = 250,000,000	150.750920	0.3820	88.05%
t = 275,000,000	150.883409	0.3788	93.21%
t = 300,000,000	150.286058	0.3931	94.90%
t = 325,000,000	150.604607	0.3855	94.15%
t = 350,000,000	150.688708	0.3835	95.00%
t = 375,000,000	150.933524	0.3776	95.92%
t = 400,000,000	150.707718	0.3830	96.56%
t = 425,000,000	150.703109	0.3831	98.21%
t = 450,000,000	150.822349	0.3803	98.94%
t = 475,000,000	150.849999	0.3796	98.77%

```

t = 500,000,000 | 150.799884 | 0.3808 | 98.29%

EVALUATION for T_eval = 100,000 periods...
Evaluation time: 0.01s

===== Results (I=2, xi=500, method='separate') =====
chi^N = 166.666633 chi^M = 124.999950
--- Per-speculator OLS (primary, Dou et al. IA.4.4) ---
Spec 0: chi_hat = 152.817302 (intercept mismatch = 0.249%)
Spec 1: chi_hat = 153.137686 (intercept mismatch = 0.166%)
bar_chi^C (OLS) = 152.977494
Speculator asymmetry = 0.209% of mean
Delta_chi (OLS) = 0.3285 (0=Nash, 1=cartel)
--- Per-period ratio (diagnostic only) ---
bar_chi^C (ratio) = 150.634222
Delta_chi (ratio) = 0.3848
--- Convergence ---
Cells stable >= 10^6 periods : 98.29%

===== seed_train=2026 =====
===== Q-learning: I=2, xi=500, method='separate', T=500,000,000 =====

Benchmarks (xi=500, I=2):
chi^N = 166.666633
chi^M = 124.999950
lambda^N = 2.000000e-03
lambda^M = 2.000001e-03

JIT warm-up (T=100)...
Warm-up done in 0.0s

TRAINING for T = 500,000,000 periods...
Training time: 76.7s (1.28 min)
t | diag_chi | Delta | frac>=1e6
t = 25,000,000 | 147.073500 | 0.4702 | 8.06%
t = 50,000,000 | 147.719813 | 0.4547 | 9.56%
t = 75,000,000 | 148.455413 | 0.4371 | 28.45%
t = 100,000,000 | 148.916818 | 0.4260 | 46.65%
t = 125,000,000 | 149.716358 | 0.4068 | 60.48%
t = 150,000,000 | 150.292971 | 0.3930 | 63.42%
t = 175,000,000 | 150.863248 | 0.3793 | 74.08%
t = 200,000,000 | 150.081565 | 0.3980 | 87.35%
t = 225,000,000 | 150.731335 | 0.3824 | 84.19%
t = 250,000,000 | 150.896082 | 0.3785 | 89.21%
t = 275,000,000 | 150.777994 | 0.3813 | 89.82%
t = 300,000,000 | 150.939284 | 0.3775 | 91.77%
t = 325,000,000 | 150.867856 | 0.3792 | 95.65%
t = 350,000,000 | 151.131681 | 0.3728 | 95.90%
t = 375,000,000 | 150.890897 | 0.3786 | 96.84%
t = 400,000,000 | 150.726727 | 0.3826 | 96.18%
t = 425,000,000 | 150.816013 | 0.3804 | 98.69%
t = 450,000,000 | 150.715782 | 0.3828 | 97.44%

```

```

t = 475,000,000 | 151.185252 | 0.3716 | 93.35%
t = 500,000,000 | 151.024538 | 0.3754 | 97.23%

EVALUATION for T_eval = 100,000 periods...
Evaluation time: 0.01s

===== Results (I=2, xi=500, method='separate') =====
chi^N = 166.666633   chi^M = 124.999950
--- Per-speculator OLS (primary, Dou et al. IA.4.4) ---
Spec 0: chi_hat = 154.161269 (intercept mismatch = 0.005%)
Spec 1: chi_hat = 152.913143 (intercept mismatch = 0.016%)
bar_chi^C (OLS)      = 153.537206
Speculator asymmetry = 0.813% of mean
Delta_chi (OLS)      = 0.3151   (0=Nash, 1=cartel)
--- Per-period ratio (diagnostic only) ---
bar_chi^C (ratio)    = 150.564240
Delta_chi (ratio)    = 0.3865
--- Convergence ---
Cells stable >= 10^6 periods : 97.23%

===== seed_train=7777 =====
===== Q-learning: I=2, xi=500, method='separate', T=500,000,000 =====

Benchmarks (xi=500, I=2):
chi^N      = 166.666633
chi^M      = 124.999950
lambda^N   = 2.000000e-03
lambda^M   = 2.000001e-03

JIT warm-up (T=100)...
Warm-up done in 0.0s

TRAINING for T = 500,000,000 periods...
Training time: 76.8s (1.28 min)
      t | diag_chi | Delta | frac>=1e6
t = 25,000,000 | 147.008407 | 0.4718 | 8.21%
t = 50,000,000 | 147.547002 | 0.4589 | 9.19%
t = 75,000,000 | 148.616703 | 0.4332 | 29.50%
t = 100,000,000 | 148.398961 | 0.4384 | 50.55%
t = 125,000,000 | 149.729607 | 0.4065 | 49.10%
t = 150,000,000 | 150.205989 | 0.3951 | 59.13%
t = 175,000,000 | 150.080989 | 0.3981 | 72.15%
t = 200,000,000 | 150.688132 | 0.3835 | 74.52%
t = 225,000,000 | 150.763593 | 0.3817 | 92.06%
t = 250,000,000 | 150.536635 | 0.3871 | 90.39%
t = 275,000,000 | 150.721542 | 0.3827 | 87.61%
t = 300,000,000 | 150.788939 | 0.3811 | 87.06%
t = 325,000,000 | 151.104031 | 0.3735 | 92.32%
t = 350,000,000 | 150.976151 | 0.3766 | 94.73%
t = 375,000,000 | 150.577533 | 0.3861 | 92.77%
t = 400,000,000 | 151.059100 | 0.3746 | 96.89%
t = 425,000,000 | 150.938708 | 0.3775 | 97.05%

```

```

t = 450,000,000 | 150.915667 | 0.3780 | 96.40%
t = 475,000,000 | 151.035483 | 0.3751 | 96.60%
t = 500,000,000 | 150.892049 | 0.3786 | 97.85%

```

EVALUATION for T_eval = 100,000 periods...

Evaluation time: 0.01s

==== Results (I=2, xi=500, method='separate') =====

chi^N = 166.666633 chi^M = 124.999950

--- Per-speculator OLS (primary, Dou et al. IA.4.4) ---

Spec 0: chi_hat = 153.435298 (intercept mismatch = 0.033%)

Spec 1: chi_hat = 153.218410 (intercept mismatch = 0.180%)

bar_chi^C (OLS) = 153.326854

Speculator asymmetry = 0.141% of mean

Delta_chi (OLS) = 0.3202 (0=Nash, 1=cartel)

--- Per-period ratio (diagnostic only) ---

bar_chi^C (ratio) = 150.175329

Delta_chi (ratio) = 0.3958

--- Convergence ---

Cells stable >= 10^6 periods : 97.85%

==== Summary =====

seed	bar_chi^C	Delta_chi	frac_stable
610014	153.8731	0.3070	96.50%
42	153.9313	0.3056	92.63%
123	152.9775	0.3285	98.29%
2026	153.5372	0.3151	97.23%
7777	153.3269	0.3202	97.85%

Delta_chi: mean = 0.3153, std = 0.0085

95% CI (assuming normality): [0.3079, 0.3227]